claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_47057438-9c9\agent-a8362c56b32d882b3.jsonl`
- SHA-256: `5d1aeddd645464f38a33d47ae89657d423d4f460c732d3b88c10d41fc131a140`
- Source modified: `2026-06-16T03:54:42+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_47057438-9c9`
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`

Transcript

1. user (2026-06-16T03:50:51.792Z)

```
You are the lead architect. Merge the three design components and the legal memo into ONE
unified 'non-attributable, tiered, encrypted-where-needed' fixture architecture (the owner
jokingly called it a 'Byzantine solution' ? make it elegant, not baroque). Deliver: a threat-
model table (who/what's protected and what is NOT); the TIERED model (public anonymized Tier-0
vs key-gated Tier-1); the concrete anti-attribution measures; a protobuf schema sketch; how
serialization stays deterministic cross-OS; the encryption mechanism + key management + CI
integration; the end-to-end contributor-on-another-machine workflow; how it plugs into the
EXISTING eval infra (name the modules to extend); explicit HONEST LIMITS (what a green suite
does and does NOT prove); a phased rollout where each phase is a SMALL PR; and the open
questions that need an OWNER (and/or counsel) decision. Build on existing infra; do not
reinvent.

DESIGN COMPONENTS (JSON):
[
  {
    "component": "Threat models + the non-attributability / minimization layer for publicly
committed video-free rally-regression fixtures",
    "summary": "Designs the de-linkage layer that sits between fusion_golden.py's
*_golden_features.json (which today carries `name` = source filename, the MD5 video_id
elsewhere, absolute match timestamps, and an aggregate `person` block) and a PUBLIC commit. Five
threat models are enumerated with a per-actor achievable/not-achievable line that respects the
lead's DRM/analog-hole limit (a test-runner can read whatever the test reads, so secrecy-from-
contributor is impossible; the goal is non-attributability + uselessness-in-isolation). The
concrete scheme: (1) opaque SALTED, non-reversible surrogate fixture IDs replacing both `name`
and the MD5 video_id; (2) strip ALL source-attribution fields (filename, video_id,
match/event/player metadata, capture dates); (3) per-fixture TIME-ORIGIN RESET that is provably
zero-cost because the metrics harness (backend/eval/metrics.py) is translation-invariant (IoU
and start/end errors are pure differences); (4) STREAM MINIMIZATION publishing only the shuttle
block + abstract relative intervals + GT, while EXCLUDING the `person` block and any minors per
the legal memo; (5) the uselessness-in-isolation argument; (6) encryption as a barrier against
the no-key public ONLY. Every choice is tied to a specific clause of the legal memo (EDPS v. SRB
relativity, the fingerprinting/video_id leak red flag, the publish-by-stream verdict, the
provenance gate, the minors flip, the de-attribution condition).",
    "threatModelsCovered": [
      "TM1 ? Public repo browser with NO key (no decryption key, no source videos): can clone
the repo and read whatever is committed in cleartext, plus anything they can brute-force.
ACHIEVABLE for them: read shuttle (x,y)-derived feature columns, abstract relative rally/GT
intervals, and run the regression harness if the fixtures are committed in plaintext. NOT
ACHIEVABLE if the design holds: (a) map any fixture back to a source video, match, event, date,
or player ? every attribution field is stripped and the id is an opaque salted surrogate, not
the MD5 video_id; (b) decrypt fixtures if the encryption-as-public-barrier option is used and
they lack the key (this is the ONE actor encryption genuinely defends against ? memo:
'encryption useful ONLY as a barrier against the no-key public'); (c) confirm a source even if
they independently obtain a candidate video, because the committed records carry no
```

MD5/timestamp/name to correlate against (defeats the memo's 'FINGERPRINTING / video_id leak' red flag). Their data is useless-in-isolation: numbers with no schema-external meaning and no source pointer.",

"TM2 ? Trusted contributor WITH the key running tests locally (the intended user): the DRM/analog-hole actor. ACHIEVABLE for them (and UNPREVENTABLE, per the lead's established constraint): read the decrypted plaintext fixtures, read the key on their own machine, read every value the regression suite reads, re-run fusion_compare/ablation. Secrecy-FROM this actor is impossible and the design does NOT pretend otherwise. NOT ACHIEVABLE even for them, because it is removed at the source (not gated by a key they hold): recover the source filename, the MD5 video_id, the match identity, the absolute match timeline, a face crop, a pose/gait track, or a named-player-to-movement mapping ? none of these are ever written into the committed fixture in the first place. So even a fully-trusted, fully-keyed contributor cannot attribute or re-identify, because attribution data is minimized away before commit, not encrypted. This is the core insight: non-attributability is achieved by DELETION + RELATIVIZATION upstream of encryption, so it survives the analog hole.",

"TM3 ? CI runner (GitHub Actions / automated harness, no human, no source videos): functionally a keyless-or-keyed automaton. ACHIEVABLE: decrypt (if the CI secret holds the key) and run the suite headlessly against committed fixtures; emit pass/fail + P/R deltas vs eval_baselines/. NOT ACHIEVABLE / NOT NEEDED: CI never needs source video, match identity, or person/biometric columns to score interval-IoU regressions, so none are present; a leaked CI log or artifact cannot leak attribution it never had. Design implication: the published fixture subset must be self-sufficient for the harness (relative intervals + shuttle features + GT) so CI is green WITHOUT any of the stripped/excluded fields ? proven runnable because fusion_compare.load_videos only consumes c['shuttle'], c['start']/c['end'], and d['gts'], and uses d['name'] solely as an opaque label.",

"TM4 ? Malicious contributor (has clone + key + intent to harm; may also fetch public videos and run our own deterministic detector). The strongest insider threat short of holding the source corpus. ACHIEVABLE: everything TM2 can do, plus actively attempting correlation attacks ? re-running the (committed, MIT/Apache) WASB/TrackNet detector on candidate public videos to produce comparable trajectories and trying to match them to a committed fixture (the de Montjoye / trajectory-linkage fingerprinting vector the memo flags). NOT ACHIEVABLE if the provenance gate + minimization hold: a successful correlation match requires a correlatable REFERENCE to exist. Because the committed subset is gated to Fork-A (owner-recorded, UNPUBLISHED source ? memo provenanceFork + conditionsForSafePublicCommit), no public reference video of that source exists, so external source-linkage is 'not reasonably likely' (memo: private/owner-recorded source). Even if they fabricate a near-match by luck, the fixture carries no timestamp/name/video_id to CONFIRM it, and the published feature set (5 fps/resolution-invariant shuttle scalars per window) is too coarse to uniquely back-match a specific 4K source. Residual risk explicitly acknowledged: this protection is conditional + revocable (memo OVERCONFIDENCE TRAP #4 / EDPS v. SRB) ? publish or leak the Fork-A source later and the link can snap back; hence the forward-looking re-test duty is part of the posture, not a one-time clearance.",

"TM5 ? Re-identification adversary running the same detector on public videos and correlating (the memo's named adversary; overlaps TM4's capability but is OUTSIDE the project, no key needed for the attack itself). ACHIEVABLE in principle: a per-video trajectory/feature stream is a fingerprint (de Montjoye: 4 points re-identify 95%; k-anonymity collapses on rich quasi-identifiers), and a deterministic detector re-run can correlate-match a stream to its source IF a correlatable public reference exists. NOT ACHIEVABLE against the committed subset BECAUSE: (a) source is Fork-A/unpublished, so no permanent cheap public reference exists to correlate against ? re-linkage is not 'reasonably likely' (the memo's precise standard); (b) the fixtures are heavily minimized to ~5 invariant shuttle scalars per window + relative intervals, far below the raw per-frame (x,y) trajectory that makes trajectory-linkage work ? the high-resolution fingerprint (the raw CSV) is NEVER committed; (c) absolute timestamps that would align a fixture to a known match broadcast/clock are reset to a relative origin. If a Fork-A source were ever published, this actor's attack becomes live ? which is exactly why provenance gating + the 'data-environment grows' re-assessment (NIST SP 800-188 / ICO motivated-intruder / OVIC forward-looking) are load-bearing, not optional."

],

"whatItAchieves": "Within the conditions below, the committed fixtures become (i) NON-ATTRIBUTABLE to a specific source video/match/event/date/person by any actor ? keyed or keyless, trusted or malicious ? because every attribution vector is removed at the source rather than merely encrypted, so it survives the DRM/analog-hole limit; and (ii) USELESS-IN-ISOLATION to a no-key public ? opaque numbers with no schema-external meaning, no source pointer, and (optionally) encrypted so the keyless public cannot even read them. Concretely it lets an external contributor on another clone run the rally-segmentation regression suite (fusion_compare/ablation LOVO; and, once predictions are produced, regression.py against

eval_baselines/) from committed structured fixtures with NO video present, while keeping the published subset on the legal memo's 'PUBLISH-OK' side of the publish/no-publish line. The time-origin reset is a free win: backend/eval/metrics.py scores via temporal IoU and |start-end| errors, both pure differences, so a per-fixture constant offset leaves every regression metric bit-identical while destroying match-timeline correlatability ? verified against metrics.py lines 16-28 and 124-125.",
    "whatItDoesNotAchieve": "It does NOT make the data 'anonymous' in an absolute sense, and the design must never claim so (memo OVERCONFIDENCE TRAP #4 / EDPS v. SRB, CJEU C-413/23 P): to the OWNER who can re-run the detector on the still-private source, the streams REMAIN personal data ? non-attributability is a property of the (data, holder, environment) triple, and the protection here is RELATIVE (to outsiders lacking the source), CONDITIONAL (on Fork-A source staying unpublished), REVOCABLE (publish/leak the source and the link snaps back), and FORWARD-LOOKING (must be re-assessed as the public-video environment grows). It does NOT defeat a contributor who runs the tests (impossible by the lead's constraint) ? encryption is explicitly NOT represented as protecting against them. It does NOT cure provenance: a single Fork-B (broadcast/YouTube/scraped) record contaminates the public commit regardless of de-attribution (memo MIXED-PROVENANCE red flag), so the scheme is necessary-but-not-sufficient and must sit BEHIND a per-record provenance gate. It does NOT, on its own, resolve the copyright-of-extraction, ToS/contract, EU-UK sui-generis database-right, or DPDP/GDPR controller-side questions ? those require the conditionsForSafePublicCommit and retained-counsel sign-off, which this design treats as gating preconditions, not as things it satisfies. It cannot publish the `person` block or any pose/gait/face stream safely (those are excluded, not protected).",
    "mechanism": "PIPELINE: fusion_golden.py (GPU box) -> a NEW de-attribution/minimization pass -> public commit. The publisher pass (a small tool, e.g. backend/eval/publish_fixtures.py or scripts/deattribute_golden.py, default-OFF of any path that touches video) transforms each *_golden_features.json into a committable *_public_fixture.json. STEP 1 ? OPAQUE SALTED SURROGATE ID: replace BOTH the `name` field (today = source filename via _derive_video_path, the .MP4/.rallies.csv stem) AND any MD5 video_id with fixture_id = first-16-hex of HMAC-SHA256(key=PER-RELEASE-SECRET-SALT, msg=video_id || name). Salt is generated once per public release, kept PRIVATE (never committed), so the surrogate is (a) non-reversible without the salt, (b) NOT the MD5 video_id (so holding the candidate file cannot confirm the source ? kills the memo 'video_id leak' red flag), and (c) unlinkable across releases. fusion_compare.load_videos uses d['name'] ONLY as a VideoCandidates label, so aliasing it is harness-transparent (verified at fusion_compare.py:50). STEP 2 ? METADATA STRIP: drop every source-attribution field ? filename, video_id, match/event/tournament/player identity, capture date/clock. (Today the JSON only carries `name`+`fps`+`frame_width`; `fps`/`frame_width` are kept since shuttle features are already fps/resolution-invariant and these are non-identifying scale constants ? but if a known roster makes frame_width a venue tell, quantize it.) STEP 3 ? TIME-ORIGIN RESET: for each fixture compute t0 = min over all candidate.start and gts; subtract t0 from every start/end and every gts pair (or add a random per-fixture offset). Proven lossless because metrics.py IoU = (min(ends)-max(starts))/union and errors = |pred-gt| are translation-invariant (metrics.py:16-28, 124-125). This severs the absolute match timeline that would align a fixture to a known broadcast. STEP 4 ? STREAM MINIMIZATION (publish-by-stream, straight from the memo's privacyVerdict): KEEP the `shuttle` block (duration, peak_velocity, mean_velocity, active_ratio, net_crossings ? inanimate-object, fps/res-invariant), the relative `start`/`end` intervals, `gts`, and `label`. EXCLUDE the entire `person` block (players_in_court, two_sided_motion, mean_player_motion, in_court_ratio, shuttle_player_colocation) from the public subset ? the memo classes per-player person tracking as the Art.9 'tripwire' and this architecture is 'PRONE to that flip', and in a small known roster aggregate person features can single out a player (DPDP). NEVER write pose/skeleton/gait/face (already absent from this schema ? gait is the memo's hard no-publish, 80-95% re-ID through anonymization). STEP 5 ? PROVENANCE GATE (precondition): only Fork-A records (owner-recorded, non-event-ticketed, minors-EXCLUDED, biometric-EXCLUDED), tagged with a verified per-record provenance flag at ingestion, are eligible; Fork-B-tainted records stay private. STEP 6 ? ENCRYPTION-AS-PUBLIC-BARRIER (optional, honest): if used, encrypt the committed blob with a key shipped to contributors/CI; documented EXPLICITLY as defending ONLY against the no-key public (TM1), never against a running contributor (TM2/TM4) ? the plaintext is non-attributable on its own, so encryption is belt-not-parachute. USELESS-IN-ISOLATION follows from 1-4: a no-key/keyed public holds opaque-id, origin-reset, source-pointer-free shuttle scalars whose only meaning is 'inputs to OUR regression harness + schema' ? no imagery, no source, no timeline, nothing that singles out a person.",
    "tradeoffs": [
      "Salt secrecy is the linchpin and the single point of failure: the surrogate fixture_id is non-reversible ONLY while the per-release salt stays private. If the salt is ever committed/leaked alongside the public fixtures, an adversary holding candidate files can recompute HMAC(salt, video_id) and confirm sources ? re-opening the very 'video_id leak' the

design closes. Mitigation: salt lives with the private Fork-A corpus (OneDrive, per GOLDEN_DATA_SHARING.md), never in git; rotate per release so a single leak does not retro-link all releases.",
      "Excluding the `person` block reduces the public suite's coverage: the multi-signal-fusion / person-cue ?F1 litmus (the Gate-A-adjacent ablation) cannot be reproduced by external contributors from the public subset ? only the shuttle-only regression is publicly runnable. This is a deliberate privacy-over-completeness trade dictated by the memo (person tracking = Art.9 tripwire + DPDP singling-out in a small roster). The person-bearing *_golden_features.json stay PRIVATE (status quo per GOLDEN_DATA_SHARING.md), so the internal litmus is unaffected; only the public mirror is thinner.",
      "Non-attributability is conditional and revocable, not permanent (EDPS v. SRB): the whole posture rests on the Fork-A source staying UNPUBLISHED. The day the owner publishes a highlight reel of that source (a stated product/paper aim), a public reference appears and TM5's correlation attack becomes live for those records. Accepting this means committing to a forward-looking re-assessment duty (NIST SP 800-188 / ICO / OVIC) and to NOT publishing the source video and its derived fixtures from the same match without re-clearing.",
      "Provenance gating shifts cost to ingestion: every golden record needs a VERIFIED per-record provenance + minors flag (memo MIXED-PROVENANCE red flag ? a single Fork-B/minor record contaminates the commit). That is process discipline the current pipeline does not yet enforce; the de-attribution pass should HARD-FAIL (refuse to emit a public fixture) on any record lacking an explicit owner-recorded + minors-excluded + biometric-excluded tag, so the gate is mechanical, not trust-based.",
      "Time-origin reset is free for the CURRENT harness (translation-invariant IoU) but constrains future metrics: any future scorer that uses absolute wall-clock time, cross-fixture temporal alignment, or duration-since-match-start would break or silently mis-score on origin-reset fixtures. Acceptable now (no such metric exists in metrics.py), but must be flagged so a future absolute-time metric is not added against public fixtures without revisiting this.",
      "Encryption adds key-distribution + false-security risk for near-zero attribution benefit: it defends only TM1, and the memo's counselQuestions flag a possible DPDP 'reasonable-security-safeguards' / misrepresentation question if encryption is marketed as protection it does not provide. Recommend treating encryption as OPTIONAL and documenting its exact (no-key-public-only) scope; do not let it become a reason to relax the upstream minimization that does the real work.",
      "fps/frame_width retained for harness scale-normalization are low-risk but not zero: in a small known squad/venue, frame_width (capture resolution) plus rally cadence could become a weak quasi-identifier. Cheap mitigation (quantize/bucket frame_width, or drop it and bake resolution-invariance fully into the published shuttle scalars) is available if counsel flags the small-roster singling-out risk."
    ],
    "integrationWithExistingInfra": "Slots into the EXISTING video-free seam without disturbing it. (1) The de-attribution pass is a NEW transform that consumes fusion_golden.py's private *_golden_features.json (on the GPU box / OneDrive per docs/GOLDEN_DATA_SHARING.md #175) and emits a PUBLIC *_public_fixture.json subset. The private features stay private and OUT of git exactly as today; only the de-attributed, minimized, Fork-A subset is committed. (2) fusion_compare.load_videos (backend/eval/fusion_compare.py:35-52) and ablation.py already read ONLY c['shuttle'][...], c['start']/c['end'], d['gts'], and use d['name'] as an opaque label ? so swapping `name` for the surrogate id and dropping the `person` block requires that the public-subset loader iterate only SHUTTLE_FEATURE_NAMES (skip PERSON_FEATURE_NAMES) for public fixtures; this is a small, additive code path, and the existing skipif litmus tests (tests/test_ablation.py::test_litmus_reproduces_gate_a, tests/test_fusion_compare.py) continue to run against the PRIVATE person-bearing features unchanged. (3) Placement follows the established precedent: headline regression baselines already live committed in eval_baselines/ (provenance-agnostic aggregate JSON snapshots ? regression_baseline.json, heuristic_lovo_n6.json, gemini_wrapper, experiment_leaderboard); the public per-fixture shuttle subset is the natural sibling under a tracked path (e.g. eval_baselines/public_fixtures/ or a new fixtures dir), while *_golden_features.json remain gitignored/private per GOLDEN_DATA_SHARING.md §Guardrails. (4) The video_id helper (backend/utils/hashing.py::compute_video_id) is the canonical MD5 source ? the de-attribution pass consumes it ONLY as private HMAC input and must guarantee it never reaches a committed file. (5) The separate concurrent workflow designing the general video-free fixture MECHANICS owns the runnable-without-video harness; THIS layer is the LEGAL/privacy filter that decides which fields of that fixture are publishable and under what id ? the two compose: mechanics produce the runnable fixture, this layer de-attributes + minimizes it before the public commit. (6) Honors project guardrails directly: MIT/Apache detectors only (so TM4/TM5 re-running 'our' detector uses a licensed stack), EXCLUDE MINORS (provenance gate hard-fail), gait/biometrics

```
strictly FUTURE (person block excluded from the public subset, pose/gait never emitted). Counsel
sign-off (memo) remains a gating precondition before the first public commit; this design
produces the de-attributed subset + provenance log that counsel reviews."
  },
  {
    "component": "Encryption / \"secret file\" layer and key management for committed rally-
segmentation regression fixtures ? a tiered (Tier-0 public/unencrypted, Tier-1 encrypted-in-repo
or out-of-repo) model with clean key-absent skips.",
    "summary": "Design a TWO-TIER fixture model layered onto the existing video-free seam
(output/*_golden_features.json re-scored by backend/eval/fusion_compare.py and
backend/eval/ablation.py; committed aggregate baselines in the tracked eval_baselines/). Tier-0
= a small, PUBLIC, UNENCRYPTED, de-attributed, Fork-A-only, biometric-free fixture set committed
under eval_baselines/fixtures/ that ANY cloner runs with NO key, giving real characterization-
regression coverage of the pure-logic re-scoring path (LOVO ?F1, bootstrap CI, sign test, the
Gate-A litmus shape). Tier-1 = richer/more-sensitive fixtures (full n=6+ golden corpus,
person/audio columns, anything Fork-B-tainted or identity-adjacent) kept OUT of the public
plaintext ? either encrypted-in-repo (SOPS+age) OR, preferred, out-of-repo on the existing
OneDrive private store fetched with a token ? for trusted contributors + CI who hold the key. A
Tier-1 test SKIPS CLEANLY (never fails) when the key/data is absent, reusing the proven
skipif(_golden_present()) pattern already in tests/test_ablation.py and
tests/test_fusion_compare.py. RECOMMENDATION: SOPS+age for any encrypted-in-repo secrets (per-
recipient public-key encryption, git-diff-friendly, value-level for JSON, trivial CI/contributor
key handling, no smudge/clean daemon), and out-of-repo+token as the DEFAULT for Tier-1 feature
corpora (keeps git lean, sidesteps the legal \"redistribution\" act entirely). git-crypt and
git-lfs+encryption are rejected for the reasons below. PROTOBUF IS SERIALIZATION, NOT SECRECY;
and Tier-1 encryption CANNOT hide content from a key-holder who runs the tests ? it is only a
barrier against the no-key public, consistent with the lead's established DRM/analog-hole
constraint.",
    "threatModelsCovered": [
      "NO-KEY PUBLIC CLONER (primary adversary the encryption layer addresses): a stranger
clones the public repo, holds no key, and tries to (a) attribute any committed fixture to a
source video/match/person, or (b) get a usable rich-feature corpus. MITIGATED for Tier-1 by
encryption-at-rest or out-of-repo storage (they see only ciphertext or a 404), and for Tier-0 by
de-attribution + uselessness-in-isolation (plaintext is readable but carries no source pointer
and is a deliberately minimal synthetic-or-Fork-A characterization set).",
      "SOURCE RE-LINKAGE / FINGERPRINTING adversary: someone who already holds a candidate video
re-runs the deterministic detector to correlate a committed trajectory/feature stream back to
its source (per the legal memo's fingerprint analysis; de Montjoye 4-points-=-95%). MITIGATED by
(i) stripping the video_id (MD5 of the source file) and every source-identifying field, (ii)
opaque random non-reversible surrogate record ids, (iii) gating committed plaintext to Fork-A
(owner-recorded, unpublished source) so no public reference exists to correlate against, and
(iv) excluding raw per-frame trajectories from Tier-0 (commit abstract rally intervals +
aggregate feature columns, not the frame-by-frame (X,Y) fingerprint).",
      "MALICIOUS-OR-CARELESS CONTRIBUTOR with the key: explicitly NOT defended against for
content secrecy (impossible per the lead's DRM/analog-hole constraint ? a contributor who runs
the tests reads plaintext + key locally). The design instead bounds the BLAST RADIUS: Tier-1
contains only de-attributed, minors-excluded, biometric-free data so even a leak exposes non-
attributable numbers, not a roster, faces, gait, or source files. Key rotation (below) handles a
known key compromise.",
      "KEY-ABSENCE / FALSE-GREEN: a CI run or contributor without the Tier-1 key must SKIP
Tier-1 cleanly, never silently pass a regression as green nor hard-fail the suite. Covered by
the skipif pattern; additionally a separate Tier-0 job runs key-free on every PR so coverage
never depends on a secret being present.",
      "ACCIDENTAL PLAINTEXT COMMIT of a Tier-1 secret (the most likely real-world failure): a
pre-commit guard + CI scan blocks committing a decrypted *_golden_features.json or any file
carrying a name/video_id/match-metadata field into a tracked path.",
      "LEGAL REDISTRIBUTION exposure (from the memo): committing Fork-B-tainted or attributable
derived data publicly is itself the high-risk 'redistribution' act remediable by injunction +
forced deletion. Covered structurally by the provenance gate (only Fork-A, de-attributed data is
ever eligible for the public/Tier-0 plaintext) and by keeping Tier-1 out-of-repo so the public
repo never carries the sensitive bytes at all."
    ],
    "whatItAchieves": "\"1) ANY cloner with NO key can run a meaningful slice of the rally-
segmentation regression suite from committed Tier-0 fixtures ? exercising the exact pure-CPU re-
scoring code paths (fusion_compare.load_videos + experiment.compare LOVO; ablation.run_ablation;
```

regression.regress against a committed baseline) that today SKIP everywhere because the golden JSONs are private. This converts the currently-skipped litmus into a real, public, characterization-level gate. 2) Trusted contributors + CI who hold the key run the FULL Tier-1 corpus for the authoritative numbers (the n=6+ ?F1/CI/sign-test, the Gate-A litmus exact values in test_litmus_reproduces_gate_a). 3) Tier-1 absence degrades gracefully to a clean SKIP, so a fresh clone is always green and the suite never depends on a secret to pass. 4) Committed plaintext is NON-ATTRIBUTABLE (no source filename, no video_id MD5, no match/player metadata, opaque surrogate ids) and USELESS-IN-ISOLATION (abstract intervals + feature columns that mean nothing without the regression harness + schema). 5) Encryption gives a genuine at-rest barrier against the no-key public for Tier-1-in-repo, and out-of-repo+token avoids putting the sensitive bytes in git history at all. 6) The model is provenance-gated so it can only ever publish Fork-A, minors-excluded, biometric-free data, keeping the public commit on the clean side of the legal memo.\"",

"whatItDoesNotAchieve": "\"1) It does NOT hide Tier-1 content from a contributor who runs the tests ? that is impossible (the lead's established DRM/analog-hole limit: running the test necessarily exposes plaintext fixtures and any decryption key on that machine). Encryption here is ONLY a barrier against the no-key public; it must never be represented as protecting against a running key-holder (a misrepresentation point the legal memo flags as a DPDP 'reasonable security safeguards' / false-security question for counsel). 2) Protobuf (or any serialization ? JSON, msgpack, Avro) provides ZERO secrecy: it is an encoding, trivially decoded by anyone with the .proto or even without it (protoc --decode_raw); never treat a binary fixture format as a confidentiality control. 3) It does NOT make the underlying data legally publishable on its own ? the provenance gate, minors exclusion, biometric exclusion, and counsel sign-off (all per the legal memo) are PRECONDITIONS the encryption layer assumes, not substitutes for. Encryption of Fork-B-tainted data does not cure the redistribution problem; it only hides bytes, while the act of distributing the encrypted blob to key-holders is still distribution. 4) It does NOT defeat a source-re-linkage adversary who already holds the source video and a published stream ? only keeping the source private (Fork-A unpublished) or coarsening/perturbing the stream does that; encryption is irrelevant once a key-holder has decrypted. 5) It does NOT achieve true anonymity of the streams in the owner's own hands ? per EDPS v. SRB they remain personal data to the holder who can re-run the detector; the achievable property is relative non-attributability to outsiders, which is conditional and revocable.\"",

"mechanism": "\"TIER-0 (public, unencrypted, no key) ? committed under a NEW tracked dir eval_baselines/fixtures/ (extending the eval_baselines/ committed-precedent), each file a *_golden_features.json-SCHEMA-COMPATIBLE JSON so the EXISTING loaders read it unchanged: fusion_compare.load_videos() and ablation.load_videos() glob the dir; experiment.compare()/run_ablation() re-score on pure CPU. Contents are DE-ATTRIBUTED and minimal: the 'name' field replaced by an opaque random surrogate id (e.g. fx_0001, NOT the current adarsh_avi_singles / kushagra_singles / mahadevpura_singles strings that the already-committed eval_baselines/heuristic_lovo_n6.json leaks today ? that file is a live de-attribution bug to fix in the same change); no video_id (the MD5 fingerprint), no filename, no capture date, no match/player metadata; candidates carry abstract (start,end) intervals + the shuttle/audio/non-biometric-aggregate-person feature columns + integer label; gts as abstract intervals. Source = either (a) a curated Fork-A, minors-excluded, biometric-free real subset, OR (b) a SYNTHETIC characterization set mirroring the test_fusion_compare/_learned_corpus construction (a known separating signal) so the public test asserts the FRAMEWORK behavior (?F1>0 when a feature separates; CI brackets mean; sign test counts) without any real-corpus provenance at all. Tier-0 tests are unconditional (no skipif) and run on every PR. \\n\\nTIER-1 (richer/sensitive, key-gated) ? two delivery options, with OUT-OF-REPO as the default:\\n (A) OUT-OF-REPO + TOKEN (preferred; reuses docs/GOLDEN_DATA_SHARING.md OneDrive store): the full *_golden_features.json corpus stays in the private OneDrive folder (or an object store). Contributors/CI set RALLY_GOLDEN_DIR (the env var already proposed in #175) to a local synced/fetched path; a tiny scripts/fetch_golden.py pulls from the private store using a token (CI secret RALLY_GOLDEN_TOKEN; contributor onboarding = granted OneDrive access or a scoped read token). Nothing sensitive ever enters git history ? which also sidesteps the legal 'redistribution' act for any Fork-B/identity-adjacent data.\\n (B) ENCRYPTED-IN-REPO (SOPS+age) when in-repo provenance/versioning is genuinely wanted for Fork-A-clean-but-still-private data: store eval_baselines/fixtures-tier1/*.json.enc encrypted with SOPS using age recipients. .sops.yaml pins the creation rule (path regex -> age public keys of each trusted contributor + the CI age key). Decrypt step: sops -d into a gitignored output/ path (or RALLY_GOLDEN_DIR) before running Tier-1 tests; the decrypted plaintext is NEVER committed (pre-commit guard below).\\n\\nKEY DISTRIBUTION:\\n - CI: store the age private key (option B) or the fetch token (option A) as a CI repository secret (RALLY_AGE_KEY / RALLY_GOLDEN_TOKEN), exposed only to the Tier-1 job on trusted branches, never to fork-PR runners (so a malicious fork PR cannot exfiltrate the key). The Tier-1 CI job is gated on the secret being present; the Tier-0 job

always runs.\\n - Contributor onboarding: a new trusted contributor generates an age keypair (age-keygen), sends their PUBLIC key; a maintainer adds it to .sops.yaml and re-encrypts (sops updatekeys) ? no existing secret is reshared, each recipient decrypts with their own private key. For option A, they are granted scoped read access to the private store.\\nKEY ROTATION:\\n - Option B: rotate by editing .sops.yaml recipients (remove a departed/compromised key, add the new one) and running sops updatekeys / re-encrypt; because age is per-recipient, removing a recipient is a one-command re-key. On a suspected DATA leak, also re-extract/re-key the underlying fixtures (the legal memo's revocability point ? a leaked stream's protection 'snaps back' only if the source stays private). Option A: rotate the fetch token / revoke the share.\\nKEY-ABSENT BEHAVIOR (skip cleanly, never fail): reuse the EXISTING pattern verbatim ? tests/test_ablation.py and tests/test_fusion_compare.py already glob output/*_golden_features.json and @pytest.mark.skipif(not _golden_present(), reason=...). Generalize to a shared helper _tier1_present() that is True iff the decrypted/fetched corpus is readable (key present AND data fetched/decrypted). Tier-1 tests skip with a clear reason ('Tier-1 golden corpus absent: set RALLY_GOLDEN_DIR or provide the key'); Tier-0 tests never skip. This guarantees a no-key clone is GREEN, not red.\\nGUARDS: a pre-commit hook + CI scan reject (i) any tracked file matching *_golden_features.json outside the de-attributed Tier-0 allowlist, (ii) any committed file containing a 'video_id'/'name' that is not an fx_ surrogate / a 32-hex MD5 / known player tokens, and (iii) a SOPS-encrypted file committed in plaintext (sops --verify / the unencrypted MAC check).\"",
    "tradeoffs": [
      "SOPS+age vs git-crypt: CHOSE SOPS+age. git-crypt encrypts WHOLE files transparently via a clean/smudge filter keyed to one shared symmetric key (or GPG recipients); its diffs are opaque binary, onboarding a contributor means re-encrypting/sharing GPG access, key rotation is painful (one shared key; rotating means re-encrypting the whole repo and the old key still decrypts history), and a misconfigured checkout silently commits plaintext. SOPS+age gives per-recipient public-key encryption (add/remove a contributor by editing .sops.yaml, no reshared secret), value-level encryption that keeps JSON structure diffable in git, simple CI handling (one env-provided age key), an explicit decrypt step (no silent-plaintext-commit failure mode), and a MAC that detects tampering. age over GPG for key ergonomics (no keyring/agent ceremony). COST: an explicit `sops -d` step in the test bootstrap (vs git-crypt's transparency) ? acceptable and arguably safer.",
      "git-lfs (+encryption) vs the above: REJECTED for fixtures. git-lfs solves LARGE-binary storage, not secrecy ? vanilla LFS objects are unencrypted on the LFS server; 'LFS + encryption' means rolling your own clean/smudge crypto filter, i.e. reinventing git-crypt with more moving parts and a server dependency. Our Tier-1 feature JSONs are SMALL (the memo/sharing-doc note they are KB-scale), so LFS buys nothing. Keep LFS strictly for any future large binary if one ever appears; never as a confidentiality control.",
      "OUT-OF-REPO+token (default) vs encrypted-in-repo: out-of-repo keeps git lean, keeps ZERO sensitive bytes in immutable git history (so a future key compromise can't decrypt old commits, and the legal 'redistribution into a public repo' act never happens for sensitive data), and reuses the already-built OneDrive store + the #175 RALLY_GOLDEN_DIR seam. COST: reproducibility depends on an external store staying alive + reachable, and access provisioning is out-of-band. Encrypted-in-repo gives atomic versioning of fixtures alongside code and offline reproducibility for key-holders, at the cost of sensitive (if encrypted) bytes living forever in history and a heavier rotation story. Pick per-fixture: Fork-A-clean-and-stable -> in-repo SOPS is fine; anything Fork-B-tainted or identity-adjacent -> out-of-repo only.",
      "Tier-0 SYNTHETIC vs Tier-0 REAL-subset fixtures: synthetic (mirroring the existing _learned_corpus / _gate_videos test builders) carries ZERO provenance/privacy risk and still exercises the full re-scoring framework, but cannot catch a regression that only manifests on real-corpus value distributions. A small Fork-A real subset gives real characterization coverage but must clear the full provenance/de-attribution/minors/biometric gate + counsel sign-off. RECOMMENDATION: ship BOTH ? synthetic for the always-on framework-behavior assertions, and a tiny vetted Fork-A real subset (once counsel-cleared) for value-level characterization; keep the authoritative real numbers in Tier-1.",
      "Encryption-as-public-barrier vs honest framing: encrypting Tier-1 risks a FALSE sense of secrecy and (per the legal memo) a DPDP 'reasonable security safeguards' / misrepresentation question ? so the design DOCUMENTS, in code comments + the sharing doc, that Tier-1 crypto is a no-key-public barrier ONLY and that a running contributor necessarily holds plaintext+key. Stating this plainly is a deliberate tradeoff against the temptation to over-claim protection.",
      "Strip-all-attribution vs keep-some-for-debuggability: removing the human-readable 'name' (adarsh_avi_singles etc.) hurts at-a-glance test debugging (a failing fx_0003 is less informative than a named video). Tradeoff resolved in favor of de-attribution: keep a PRIVATE surrogate->source map in the out-of-repo store for owner/CI debugging, never commit it."
    ],

```
    "integrationWithExistingInfra": "\"Builds entirely on seams that already exist ? minimal new surface:\\n1) eval_baselines/ (TRACKED, per regression.py's DEFAULT_BASELINE_PATH comment 'NOT under the gitignored output/, so a fresh checkout/CI can load a committed baseline') is the committed-structured-data precedent. Tier-0 fixtures live in a new tracked subdir eval_baselines/fixtures/ alongside the existing regression_baseline.json / experiment_leaderboard.json / heuristic_lovo_n6.json.\\n2) The video-free re-scoring loaders are reused UNCHANGED: backend/eval/fusion_compare.py::load_videos and backend/eval/ablation.py::load_videos both glob('*_golden_features.json') in a dir, so Tier-0 fixtures only need to honor that schema (candidates[].{start,end,shuttle{},person{},audio{},label}, gts, name). The schema-compatibility is the whole integration cost.\\n3) The skip-clean-when-absent contract already ships: tests/test_ablation.py::_golden_present() + @skipif and tests/test_fusion_compare.py do exactly the right thing. Generalize to a shared tests helper for Tier-1; add a NEW unconditional Tier-0 test module (tests/test_fixtures_tier0.py) that points the same loaders at eval_baselines/fixtures/ and runs on every PR (folds into run_all_tests.py, the offline fully-mocked suite).\\n4) RALLY_GOLDEN_DIR (proposed in docs/GOLDEN_DATA_SHARING.md / #175) becomes the single switch that points fusion_golden/fusion_compare/ablation at either the synced OneDrive Tier-1 corpus or a sops-decrypted local path; the scripts/sync_golden.py sketched in #175 grows a fetch/decrypt sibling (scripts/fetch_golden.py / a `sops -d` make target).\\n5) regression.py's incommensurability guards (iou/detector/gt_hash) and the experiment leaderboard's label_set_hash (an opaque SHA, already commit-safe) are unaffected and complement de-attribution ? the SHA is a non-reversible fingerprint of the LABEL SET, not of a source video, so it is fine to keep public.\\n6) IMMEDIATE companion fix in the same change: scrub eval_baselines/heuristic_lovo_n6.json ? its per_video keys (adarsh_avi_singles, kushagra_singles, mahadevpura_singles, gbaaddy, ...) are committed plaintext player/source names that violate the de-attribution rule; replace with opaque surrogate ids (and keep the mapping private). This is a concrete, already-committed leak the design must remediate, not a hypothetical.\\n7) hashing.py::compute_video_id (MD5 of source) stays an internal DB/key concept and is explicitly EXCLUDED from any committed fixture, per the fingerprint red flag in the legal memo. CI guard enforces it.\\nNet: no new registry, no schema fork, no large binary tooling ? a tracked fixtures subdir + one unconditional test module + a generalized skipif helper + a fetch/decrypt script + a pre-commit/CI attribution-leak guard, on top of the existing eval_baselines / golden-features / RALLY_GOLDEN_DIR machinery.\""
  },
  {
    "component": "Protobuf fixture SCHEMA + video-free test-harness integration (committed golden fixtures replay through the real segmenter/fusion code; characterization-snapshot + quality-floor assertions; deterministic, cross-OS-stable, legally de-attributed)",
    "summary": "A per-fixture protobuf record (proto3) that serializes EXACTLY the JSON contract `backend/eval/fusion_golden.py` already emits (`name`/`fps`/`frame_width` + per-candidate `start/end` + `shuttle{5}`+`person{5}`+`audio{3}` feature maps + `label`, plus `gts` intervals), ADDING three things the existing JSON lacks: (1) a `snapshot` of expected output segments for characterization, (2) `schema_version`/`label_version`/`detector_tag` provenance for the incommensurability guards, and (3) the legal-memo de-attribution: an opaque `fixture_id` (random surrogate, NEVER the video_id MD5), zero source filename/match/player fields, and a column allowlist that bans pose/gait/face. Generated Python lives at `backend/eval/fixtures/_gen/` (committed, never hand-edited). An auto-discovered parametrized pytest globs the committed `tests/fixtures/golden/*.pb`, deserializes each, rebuilds `experiment.VideoCandidates`, REPLAYS it through the real `experiment.predict`/`compare`/`ablation` code (the SAME video-free seam `fusion_compare.py`/`ablation.py` use), and asserts BOTH a byte-stable snapshot match (catches any refactor side-effect) AND a quality-floor vs GT (P/R/F1 >= per-fixture floor). Reuses, never reinvents: the loaders, `Variant`, `gt_hash`, `litmus_config`, `eval_baselines/` placement precedent, and the `skipif` litmus pattern.",
    "threatModelsCovered": [
      "Refactor side-effects: a snapshot/characterization assertion on the replayed output segments catches ANY behavioral drift in predict/merge/gate/scorer code, even quality-neutral ones (the gap regression.py leaves open ? it only checks P/R drop, not exact output)",
      "Quality regression: a per-fixture quality-floor assertion (P/R/F1 vs committed GT, reusing metrics.score) fails when a change degrades detection below the floor ? the video-free equivalent of regression.py's baseline check, but runnable on a fresh clone with NO video and NO DB",
      "Incommensurable comparison: schema_version/label_version/detector_tag/iou + a gt_hash (reusing regression.gt_hash) travel IN the fixture, so a fixture scored under a different label set, detector, or IoU is flagged as incommensurable rather than silently mis-compared (mirrors regression.py's existing guards)",
```

"Cross-OS / non-determinism flake: sorted fields + fixed-precision float quantization + LF newline normalization + a canonical-bytes round-trip make the serialized fixture and the snapshot byte-identical on Windows/Linux/mac, so CI cannot flake on float repr or CRLF",
"Skipped-everywhere coverage hole: the existing litmus tests (test_ablation.py / test_fusion_compare.py) skipif the private *_golden_features.json are absent, so they run NOWHERE in CI; committed .pb fixtures make the video-free seam actually exercised on every clone",
"Legal/IP + privacy exposure (per the memo): provenance gate to Fork-A only, opaque surrogate fixture_id instead of the video_id MD5 fingerprint, stripped source/match/player attribution, and a hard column-allowlist excluding pose/gait/face/minors ? so a committed fixture is non-attributable and useless-in-isolation",
"Schema evolution breakage: schema_version + an additive proto3 discipline + a tiny migration shim let old committed fixtures keep loading when columns/fields are added, so a schema bump never silently invalidates the corpus or the snapshots"
],
"whatItAchieves": "An external contributor on a fresh clone (no video, no GPU, no DB) can run the rally-segmentation regression suite against committed fixtures: the parametrized test replays each fixture through the REAL segmenter/fusion/scoring code and asserts both that the output is byte-identical to the committed snapshot (any refactor that changes behavior fails loudly) and that quality stays at/above a per-fixture floor vs ground truth. The fixtures are deterministic and byte-stable across OSes, version-guarded so they cannot be mis-compared, and ? by construction ? non-attributable to any source video/match/person and useless in isolation, satisfying the legal/privacy conditions for a public commit. It closes the two open gaps in the current infra: regression.py is NOT video-free on the prediction side (needs a pipeline run on the video), and the litmus tests are skipped everywhere because the only feature files are private OneDrive JSONs.",
"whatItDoesNotAchieve": "Does NOT make the GPU EXTRACTION step (fusion_golden.py: video+CSV -> features) video-free ? that still needs the GPU/WSL box; this only commits its OUTPUT. Does NOT replace regression.py's DB-backed, full-pipeline-on-real-video promotion gate ? it is the cheap CPU shadow that catches refactor/quality drift, not the authoritative serve-path check. Does NOT provide secrecy-FROM-the-contributor: a contributor who runs the tests can read the plaintext fixture and any key on their machine (the lead's established DRM/analog-hole limit) ? the goal is non-attributability + uselessness-in-isolation, with encryption useful ONLY as a barrier against the no-key public, never against a running contributor. Does NOT itself decide provenance/licensing/minors questions ? those are owner+counsel gates (the memo); the schema merely ENFORCES the de-attribution mechanically (opaque id, banned columns, no source fields). Does NOT cover signals beyond the persisted shuttle+person+audio+gate columns; pose/gait/face are deliberately unrepresentable in the schema. Cannot, on its own, prove the source video is Fork-A ? it relies on a verified provenance flag set at ingestion upstream.",
"mechanism": "PROTO SCHEMA (proto3, file `backend/eval/fixtures/rally_fixture.proto`, package `rally.fixture.v1`):\n\n```proto\nsyntax = \"proto3\";\npackage rally.fixture.v1;\n\n// One committed, video-free regression fixture = one (de-attributed) golden video.\nmessage RallyFixture {\n // --- versioning / incommensurability guards (mirror regression.py + ablation.py) ---\n uint32 schema_version = 1;   // bump on a non-additive proto change; gates migration\n string label_version = 2;   // Roora guideline version (e.g. \"v8\") ? the label_version axis\n string detector_tag   = 3;   // e.g. \"wasb_hybrid@<ver>\" ? the regression.py detector key\n string gt_hash   = 4;   // regression.gt_hash(gts), recomputed+verified on load\n float iou_threshold = 5;   // IoU the snapshot/floor were computed at (default 0.5)\n\n // --- de-attribution (legal memo): NO source filename / video_id MD5 / match / player ---\n string fixture_id = 6;   // OPAQUE random surrogate (uuid4 hex). NEVER the MD5 video_id.\n\n // --- replay inputs (EXACTLY today's *_golden_features.json contract) ---\n float fps   = 7;\n float frame_width = 8;\n repeated Candidate candidates = 9; // recall-oriented proposal windows + feature columns\n repeated Interval gts   = 10; // ground-truth rally intervals\n\n // --- expected outputs for the two assertions ---\n Snapshot expected = 11;   // characterization (what predict() returned when blessed)\n QualityFloor floor = 12;   // minimum P/R/F1 vs gts this fixture must hold\n}\n\nmessage Candidate {\n Interval window = 1;   // {start,end} seconds (the (start,end) interval contract)\n // Column-keyed maps mirror the shuttle{}/person{}/audio{} JSON blocks 1:1. A map (not a\n // packed array) so a new column is additive and an old fixture missing it defaults to 0.0,\n // exactly like experiment._feature_column / ablation.load_videos tolerate today.\n map<string, double> shuttle = 2;   // duration,peak_velocity,mean_velocity,active_ratio,net_crossings\n map<string, double> person = 3;   // players_in_court,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation\n map<string, double> audio   = 4;   // audio_hit_count,audio_hit_rate,audio_hit_strength_mean\n

int32 label = 5;   // IoU-derived 0/1 (label_candidates), kept for parity\n // NOTE: NO pose/skeleton/gait/face/bbox-track columns exist in this schema by design.\n}\n\nmessage Interval { double start = 1; double end = 2; }\n\nmessage Snapshot { // the blessed output of replaying this fixture\n string variant_spec_hash = 1;   // Variant.spec_hash() the snapshot was taken under\n repeated Interval segments = 2;   // sorted, quantized predicted rally intervals\n}\n\nmessage QualityFloor { float min_precision = 1; float min_recall = 2; float min_f1 = 3; }\n```\n\nCODEGEN & PLACEMENT: add `protobuf` (Apache-2.0 ? clears the MIT/Apache/BSD guardrail) to requirements. Generate with `protoc --python_out=backend/eval/fixtures/_gen rally_fixture.proto` (a `tools/gen_proto.py` wrapper + a `make proto` target so it is reproducible, pinned protoc version recorded). The generated `rally_fixture_pb2.py` is COMMITTED under `backend/eval/fixtures/_gen/` with a `# generated ? do not edit` header and an `__init__.py`; a tiny CI guard re-runs protoc and `git diff --exit-code` to prove the checked-in code matches the .proto. (Pure-Python protobuf runtime, no GPU/cv2 ? runs in the dev container.)\n\nVIDEO-FREE REPLAY (the seam): a new `backend/eval/fixtures/loader.py` mirrors `fusion_compare.load_videos` but reads `.pb`: `RallyFixture -> experiment.VideoCandidates` (map keys -> features dict, gts -> gts) ? identical downstream object, so it flows through `experiment.predict`/`compare`/`run_ablation` UNCHANGED. The fixture's `detector_tag`/`label_version`/`iou_threshold` are checked against the variant/run config using the SAME pattern as regression.py's incommensurability guards; `gt_hash` is recomputed via `regression.gt_hash(gts)` and must equal the stored value or the test errors (labels changed).\n\nAUTO-DISCOVERED PARAMETRIZED TEST (`tests/test_fixture_replay.py`): `pytest_generate_tests` (or `@pytest.mark.parametrize` over `glob('tests/fixtures/golden/*.pb')`) yields one test per committed fixture. Each test: (a) load -> VideoCandidates; (b) pick the variant from `expected.variant_spec_hash` (default = the same control/litmus Variant the snapshot was blessed under); (c) `segments = experiment.predict(variant, vc)` ? REAL code; (d) SNAPSHOT assert: `canonicalize(segments) == canonicalize(expected.segments)` (catches refactor side-effects); (e) FLOOR assert: `s = metrics.score(segments, gts, iou); assert s.precision>=floor.min_precision and s.recall>=floor.min_recall and s.f1>=floor.min_f1`. Unlike today's litmus tests this does NOT skipif ? the fixtures are committed, so it runs on every clone/CI. A `--bless` env flag (or a `scripts/bless_fixtures.py`) regenerates `expected`+`floor` when a change is intentional, the way a snapshot library blesses.\n\nDETERMINISM / CROSS-OS STABILITY: (1) FLOATS ? quantize every coordinate/feature to fixed precision before serialization (e.g. round(x, 6) for features, round(t, 3) for seconds, matching gt_hash's round(.,3)); store quantized so Windows/Linux float repr can never diverge. (2) FIELD ORDER ? sort `candidates` by (start,end), sort feature-map iteration by key, sort `segments`; serialize via a canonical writer that sets deterministic serialization on the message (protobuf map ordering is otherwise unspecified) and writes sorted. (3) NEWLINES ? fixtures are binary `.pb` (no newline issue) and committed with `*.pb binary` + `-text` in `.gitattributes` so git never CRLF-mangles them; any sidecar text (a human-readable `.txtpb`/JSON debug dump) is normalized to LF. (4) A round-trip test asserts `serialize(parse(bytes)) == bytes` for every committed fixture (canonical bytes are stable).\n\nVERSIONING & MIGRATION: `schema_version` gates a small `migrate(raw_bytes, from_v) -> current` shim in loader.py; proto3 field-additions are backward-compatible (old fixtures parse, new fields default), so a column add is additive (new map key, defaults 0.0 ? already tolerated). A non-additive change bumps `schema_version` and adds a migration branch + a re-bless of snapshots. `label_version` tracks the Roora guideline version so a relabel is detected as incommensurable (not silently re-scored), exactly as the gt_hash guard intends.",

    "tradeoffs": [

      "Protobuf vs reusing the existing JSON: JSON is already the on-disk format (*_golden_features.json) and needs no new dep, but it has no schema enforcement, no native versioning, and float/key-order are non-canonical (cross-OS flake risk). Proto buys a typed contract, codegen-enforced shape, and deterministic canonical bytes ? at the cost of a build step (protoc) and a committed generated module. Given the determinism + versioning requirements are first-class here, proto earns its keep; a JSON-Schema-validated canonical-JSON alternative would avoid the build step but reimplement determinism by hand.",

      "Committing generated *_pb2.py vs generating at build time: committing makes a fresh clone work with zero protoc install (contributor-friendly) but risks drift between .proto and generated code ? mitigated by the CI `protoc + git diff --exit-code` guard. The alternative (generate in CI/conftest) avoids committed artifacts but forces every contributor to have a pinned protoc, which contradicts the 'runs on a fresh clone' goal.",

      "Snapshot (characterization) assertion catches EVERY behavioral change including intentional improvements, so it will fail on legitimate refactors and must be re-blessed ? accepted cost: that is exactly its value (no silent drift), and a `--bless` path keeps the churn cheap. Pairing it with the quality-floor means an intentional improvement re-blesses the snapshot while the floor guarantees it didn't regress quality.",

```
      "map<string,double> feature columns vs a fixed repeated field per named feature: the map
is additive/forward-compatible (new column = new key, old fixtures default to 0.0, matching the
existing loaders' tolerance) but map iteration order is unspecified in protobuf ? paid for with
the canonical sorted writer + deterministic serialization. A rigid per-feature field list would
be self-documenting and ordered but would force a schema bump + migration for every new cue,
fighting the weak-signals-retained 'add columns freely' norm.",
      "Quality-floor as a fixed per-fixture minimum vs a baseline-delta tolerance (regression.py
style): a fixed floor is simpler and video-free (no baseline file to key by video_id, which
would re-introduce a source identifier) but is coarser than regression.py's tolerance-vs-
baseline. Chosen because the floor avoids committing a per-video baseline keyed on a source id ?
keeping the de-attribution intact ? while the snapshot covers fine-grained drift.",
      "De-attribution (opaque id, stripped source, banned columns) reduces the fixtures'
debuggability ? you cannot map a failing fixture back to its source match without the private
provenance log ? but that opacity IS the legal requirement (non-attributability); the private
OneDrive *_golden_features.json keep full attribution for owner-side debugging.",
      "Encryption-at-rest of the committed .pb is deliberately NOT in the core design (only an
optional public-barrier layer): it cannot bind a contributor who runs the tests, and adding it
risks a false sense of security and a DPDP 'reasonable safeguards' misrepresentation question
(memo counselQuestion). Better to ship plaintext-but-non-attributable fixtures than encrypted-
but-attributable ones."
    ],
    "integrationWithExistingInfra": "EXTEND, do not reinvent: (1) ON-DISK CONTRACT ? the proto
mirrors the exact `*_golden_features.json` shape from
`backend/eval/fusion_golden.py::extract_video`
(`name/fps/frame_width/candidates[{start,end,shuttle,person,audio,label}]/gts`); the only ADDED
fields are `schema_version`, `label_version`, `detector_tag`, `gt_hash`, `iou_threshold`, opaque
`fixture_id`, `expected` Snapshot, `floor`. Column names come straight from
`distill_local.FEATURE_NAMES` (shuttle 5), `fusion_features.PERSON_FEATURE_NAMES` (person 5),
`audio_features.AUDIO_FEATURE_NAMES` (audio 3) ? reuse those constants, do not re-list. (2)
LOADER ? add `backend/eval/fixtures/loader.py` that converts `RallyFixture ->
experiment.VideoCandidates`, structurally cloning
`fusion_compare.load_videos`/`ablation.load_videos` (same
VideoCandidates(name,intervals,features,gts)); the missing-audio default-to-0.0 tolerance
already exists in `experiment._feature_column` ? reuse it. (3) REPLAY ? call the unchanged
`experiment.predict`/`compare`/`run_ablation` and `metrics.score`; the litmus path reuses
`ablation.litmus_config()` so a committed fixture reproduces the Gate-A result the same way
`test_litmus_reproduces_gate_a` does today, but WITHOUT skipif. (4) GUARDS ? recompute the
gt_hash with `regression.gt_hash` and reuse regression.py's detector/IoU incommensurability
checks; reuse `Variant.spec_hash()` for the snapshot's variant key. (5) PLACEMENT ? commit the
binary fixtures under `tests/fixtures/golden/*.pb` (or, to match the committed-structured-
regression precedent, alongside `eval_baselines/`), and keep the FULL-attribution
`*_golden_features.json` PRIVATE on OneDrive per `docs/GOLDEN_DATA_SHARING.md` (#175). A small
`scripts/build_fixtures.py` converts a private Fork-A, minors-excluded, biometric-free
`*_golden_features.json` -> a de-attributed `.pb` (assigns the opaque fixture_id, strips name,
blesses snapshot+floor), so the GPU box stays the single producer. (6) TEST ENTRY ? the new
`tests/test_fixture_replay.py` is auto-collected by `run_all_tests.py`/pytest with NO new
runner. Files to touch: NEW
`backend/eval/fixtures/{rally_fixture.proto,loader.py,_gen/rally_fixture_pb2.py}`, NEW
`scripts/build_fixtures.py`, NEW `tests/test_fixture_replay.py`, NEW
`tests/fixtures/golden/*.pb`, `.gitattributes` (`*.pb binary -text`), `requirements*.txt`
(+protobuf), and a doc note in `docs/GOLDEN_DATA_SHARING.md`. Do NOT modify
`experiment.py`/`regression.py`/`ablation.py`/`fusion_compare.py` internals ? the design is
purely additive on top of their existing seams."
  }
]

LEGAL MEMO (JSON):
{
  "executiveSummary": "INTERNAL MEMO ? general legal information, NOT legal advice; retained
India + EU/UK counsel sign-off required before any public commit. QUESTION: can the project
commit DERIVED numeric data (per-frame shuttle (x,y) coordinates, rally start/end time-
intervals, audio/person/court feature columns, ground-truth intervals) into a public GitHub repo
so external contributors can run the rally-segmentation regression suite WITHOUT the videos?
BOTTOM LINE: Yes, but only under conditions, and the analysis FORKS on data provenance. The
OUTPUT data itself is, across all three jurisdictions, essentially uncopyrightable
```

FACT/measurement ? it does not inherit the source video's copyright (US: Feist + 17 USC 102(b) + NBA v. Motorola; India: EBC v. D.B. Modak, no sui generis right; EU: Football Dataco v. Yahoo ? copyright protects only original selection/arrangement, never the numbers). So a thin compilation copyright at most attaches to a creative schema, never to the coordinates. BUT copyright is the easy axis and NOT where the risk lives. The real exposure is fourfold and provenance- and stream-dependent: (1) the ACT of ingesting/extracting from the source video needs its own lawful basis ? owned/permissioned video is clean; a US fair-use defense for analytic extraction is now CONTESTED, not \"strongly favorable,\" after Thomson Reuters v. Ross (2025) and the US Copyright Office May-2025 report, and unlawful acquisition independently destroys the defense (Bartz v. Anthropic, ~$1.5B settlement); (2) CONTRACT / Terms-of-Service can bar extraction and redistribution even where copyright cannot reach the facts (ProCD; hiQ v. LinkedIn; YouTube ToS bars scraping AND \"harvest information that might identify a person (for example... faces)\" AND redistribution) ? and the remedy is injunction + forced deletion of the committed dataset, not nominal damages; (3) EU/UK sui generis DATABASE RIGHT may subsist in self-derived match-event/coordinate streams because, on the leading Football Dataco v. Sportradar reasoning, recording pre-existing on-court facts you do not control is \"OBTAINING,\" not \"creating\" ? so the project's comfortable \"we created it\" premise is genuinely contestable; (4) PRIVACY ? person/pose/gait streams and anything involving MINORS are the highest-risk and several should NOT be published at all. The cleanest defensible path: commit ONLY owned-or-permissioned-source, minors-excluded, person/pose/gait-EXCLUDED (shuttle + abstract rally-interval + non-biometric feature) data, stripped of every source-attribution field (no filename, no video_id MD5, no match metadata), under a no-extraction-from-third-party-source and no-redistribution hygiene posture, with optional encryption only as a barrier against the no-key public (it cannot bind a contributor who runs the tests).",
  "derivedDataCoreVerdict": "The derived numeric data DOES NOT inherit the source video's copyright in any of the three jurisdictions ? this is the one firmly-settled conclusion. Per-frame shuttle (x,y) coordinates, rally start/end timestamps, and audio/person/court feature values are uncopyrightable FACTS/measurements about a recorded physical event: discovered, not authored. US: Feist (facts never copyrightable; \"originality, not sweat of the brow, is the touchstone\"), 17 USC 102(b) (no copyright in any \"discovery\"), NBA v. Motorola (facts generated by a sporting event are uncopyrightable \"purely factual information which any patron... could acquire from the arena\" even though the broadcast is protected). A coordinate/timestamp table is also NOT a 17 USC 101 \"derivative work\": a derivative work recasts the original's protected EXPRESSION (framing, editing, imagery, commentary), and a number table carries none of it. India: EBC v. D.B. Modak ? copyright needs \"skill and judgment,\" protects only original arrangement, not facts; no EU-style sui generis right exists. EU: Football Dataco v. Yahoo (C-604/10) ? database copyright protects only an original selection/arrangement; \"the intellectual effort and skill of creating that data are not relevant\" and sweat-of-the-brow cannot ground it. CEILING ON PROTECTION: the MOST that can attach is a THIN compilation copyright in a genuinely creative SELECTION/ARRANGEMENT (e.g., a curated schema) ? never in the raw numbers, and a mechanical/exhaustive layout (every frame, every rally) likely has too little originality even for that (Feist's \"garden-variety listing\" problem). CRITICAL SEPARATION: the STATUS of the output data (settled: unprotected facts) is a different question from the ACT of extraction (transiently copying the video to compute the data), which can independently infringe (Sega v. Accolade: intermediate copying can infringe even when the end product reproduces no protected expression) and which needs its own license or fair-use basis. NON-COPYRIGHT FENCES survive precisely BECAUSE the data is uncopyrightable: enforceable no-extraction ToS (ProCD), narrow hot-news misappropriation (NBA v. Motorola), and the EU/UK sui generis database right. So \"the numbers are free facts\" answers the copyright question and ONLY the copyright question.",
  "byJurisdiction": [
    {
      "jurisdiction": "India (PRIMARY ? owner is Bangalore-based, commercialization intended)",
      "copyright": "Derived facts are NOT copyrightable as such: EBC v. D.B. Modak (2008 SC) requires 'skill and judgment' and protects only original arrangement, not facts; India has NO EU-style sui generis database right. BUT this is not as clean as it looks: Indian High Courts protect DATABASES/COMPILATIONS as 'original literary works' under s.2(o) on a low 'skill, labour and judgment' threshold ? Burlington Home Shopping v. Rajnish Chibber, V. Govindan v. E.M. Gopalakrishna Kone, and the OLX v. Padawan order itself (which held a scraped dataset to be a protectable 'proprietary database'). So if numbers are LIFTED from a third party's structured dataset/feed, copying its selection/arrangement can infringe even though the raw facts are free. Self-derived-from-pixels data avoids this; copying an official stats compilation does not.",
      "databaseRight": "No sui generis database right exists in India ? a genuine permissiveness advantage over the EU/UK. Protection runs only through copyright (compilation, above) or contract/IT-Act.",

```
      "contractToS": "The LIVE risk, not copyright. Breach of anti-scraping ToS plus IT Act s.43 (Government has affirmatively argued public-data scraping 'may be in breach of section 43') and s.66 layering CRIMINAL exposure (up to 3 years / Rs 5 lakh). Enforceability of anti-scraping ToS is untested in Indian courts. OLX v. Padawan was decided AGAINST the scraper; its 'private use' line is obiter, not a safe harbor. Trade-secret/confidentiality for the owner's OWN corpus runs NOT through any US-style DTSA (India has no trade-secret statute) but through the Indian Contract Act 1872, equity, common-law breach of confidence, and IT Act s.72 ? and publication still forfeits the moat under that doctrine.",
      "privacy": "DPDP Act 2023 governs personal data of identifiable natural persons; genuinely anonymized numbers fall outside it, but the s.3(c)(ii) 'publicly available data' carve-out is CONTESTED (MeitY/Minister has said scrapers must still meet DPDPA consent obligations). MINORS are stricter than the EU: 'child' = under 18 (s.2(f)); s.9(1) requires VERIFIABLE parental consent (DPDP Rules 2025 Rule 10 ? real ID/age verification, not a checkbox); s.9(2) bars detrimental processing; s.9(3) PROHIBITS tracking/behavioural monitoring/targeted ads of children ? subject to narrow Rule 12/Fourth-Schedule class/purpose exemptions that do NOT cover an amateur highlight indexer, so the ban DOES bite this project. A persistent per-player pose/gait signature is, on the conservative reading, prohibited behavioural monitoring; 'tracking' is undefined in the Act, so this is the defensible-conservative, not the certain, reading.",
      "netVerdict": "MORE PERMISSIVE than the EU on the database-right axis, but NOT 'lawful on its face.' Defensible to publish ONLY genuinely-anonymized, independently-derived (not lifted from a third-party compilation) numbers, with NO minors tracked and accepting that anti-scraping-ToS + IT Act s.43/s.66 exposure for any third-party-sourced ingestion is unresolved. Owned-source + minors-excluded + no-person/pose/gait = clean."
    },
    {
      "jurisdiction": "United States (repo + product are global; US fair-use & contract law are the most-developed)",
      "copyright": "Output data does not inherit the video's copyright (Feist; 102(b); NBA v. Motorola); not a 101 derivative work; thin compilation copyright at most. The EXTRACTION act is the open question: fair use for analytic, non-expressive copying was historically favorable (Authors Guild v. Google; Google v. Oracle) but is now CONTESTED ? Thomson Reuters v. Ross (D. Del. 2025) REJECTED the 'copy an expressive work just to reach unprotected facts/data' theory, distinguished the functional-CODE cases (Sega/Oracle), found the use non-transformative, and found Factor-4 harm in a derivative 'data to train AIs' market the plaintiff had not even entered; the US Copyright Office (May 2025) rejected 'non-expressive therefore inherently transformative' and backs a market-dilution harm theory. If the dataset competes with a rights-holder's own data/analytics market, fair use may FAIL. Unlawful source acquisition independently defeats the defense regardless of how transformative the end use is (Bartz v. Anthropic, 2025; ~$1.5B settlement).",
      "databaseRight": "No sui generis database right in the US (Feist rejected sweat-of-the-brow). Competitors stay free to copy raw facts ? subject to the non-copyright fences below.",
      "contractToS": "The dominant US risk. Post-Van Buren the CFAA no longer reaches mere purpose-violations of accessible/public data (though footnote 8 RESERVED whether contract/policy limits can still ground CFAA liability ? the federal-crime route is narrowed, not dead), so platforms enforce via CONTRACT. ToS is an enforceable contract that bites where copyright/CFAA fail (ProCD; hiQ v. LinkedIn ? breach-of-contract summary judgment + $500k consent judgment + injunction + ORDER TO DESTROY scraped-derived data). Binding turns on assent: a logged-in/account/API-key or genuine clickwrap user is bound (hiQ; any YouTube-API user); a purely logged-out scraper under browsewrap-only may not be (Meta v. Bright Data) ? and thin 'sign-in wrap' flows can fail for want of conspicuous notice. YouTube ToS verbatim bars automated access, 'harvest any information that might identify a person (for example... faces)', and redistribution/commercial reuse. REDISTRIBUTION (committing the dataset publicly) is the single highest-risk act and the remedy is injunction + forced deletion, not nominal damages.",
      "privacy": "GDPR-style analysis below applies to global users; US state biometric statutes (e.g., Illinois BIPA ? not separately briefed here) and publicity/personality rights are additional fences for face crops and identifiable-player gait. Faces in any committed imagery = identifying personal data; do not publish.",
      "netVerdict": "Output data publishable as facts ONLY if (i) source video was owned/permissioned (or extraction has a defensible fair-use basis you do NOT over-rely on), (ii) no enforceable ToS bars extraction/redistribution of the ingested source, and (iii) no person/pose/gait/face/minor stream is included. Treat third-party-ToS-sourced derived data as NOT publicly committable."
    },
    {
```

```
      "jurisdiction": "EU / UK (product/repo global; English-language highlight market makes UK
a likely target)",
      "copyright": "Database copyright protects only an original selection/arrangement, never
the numbers, and never sweat-of-the-brow (Football Dataco v. Yahoo, C-604/10). Self-derived
coordinates are facts, not a copied original structure ? so copyright is a low axis.",
      "databaseRight": "The genuinely contestable axis. The sui generis right protects
investment in OBTAINING/verifying/presenting pre-existing data, NOT in CREATING it (BHB v.
William Hill C-203/02; Fixtures Marketing) ? which the project reads as 'we create coordinates
frame-by-frame, so we are clear.' THAT READING IS OVERCONFIDENT: in Football Dataco v.
Sportradar the live match-event data (goals/scorers) was held to be OBTAINED, because the
recorder captured pre-existing facts ('events taking place on the pitch') it did NOT control. A
badminton rally/shuttle position is the same kind of pre-existing on-court fact recorded rather
than authored, so a rights-holder can argue the right SUBSISTS in our streams. Infringement is
now governed by CV-Online Latvia (C-762/19, 2021): even a substantial-part taking is actionable
only where it adversely affects the maker's ability to recoup investment, balanced against
information flow/competition ? a fact-intensive harm test, not a clean 'no protected part =
safe' rule. Sportradar (C-173/11) also defeats the server-location dodge for EU-targeted re-
utilisation. UK retained its own sui generis right and pre-2021 CJEU case law, so UK-targeted
products carry AT LEAST equal, arguably greater, risk; a non-UK/EEA maker may not even hold the
right defensively, which does nothing to reduce exposure as a potential infringer.",
      "contractToS": "Ryanair v. PR Aviation (C-30/14): where a database enjoys NEITHER
copyright NOR the sui generis right, the Directive's user-protections (incl. no-contracting-out)
do not apply, so the source's T&Cs/licence can lawfully BAN automated extraction and
redistribution and bind anyone who agreed ? the dominant residual risk. Trap: the licence that
legitimises INGESTION of the footage is often the very instrument that contractually bans
extraction/redistribution of derived data.",
      "privacy": "GDPR: shuttle/object coordinates are NOT personal data in isolation (Art.4(1))
? but coordinates published as part of an identifiable named match are part of a personal-data
operation. Person bounding boxes that single out a human are ordinary personal data, NOT Art.9
biometric UNLESS used to uniquely identify (EDPB Guidelines 3/2019). Pose/gait is the danger
zone: gait re-identifies even with faces blurred and skeleton 'anonymization' leaves gait
identity intact (Weiss et al., Sensors 2025) ? Art.9 special-category the moment used to
uniquely identify; explicit-consent basis required. 'Anonymous' is RELATIVE post EDPS v. SRB
(CJEU C-413/23 P, 4 Sept 2025): a stream can be non-attributable to a recipient lacking the
source yet remains PERSONAL DATA to the holder who can re-run the detector. Minors: Art.8
(consent floor 13?16); any biometric/special-category processing of a child stacks Art.9.",
      "netVerdict": "Do NOT assume self-derivation defeats the database right ? treat self-
derived event/coordinate streams as potentially 'obtained' data and avoid taking in a way that
could reconstitute a rights-holder's dataset or harm their feed's monetisation. Read every
source's T&Cs for extraction/redistribution clauses. Keep the footage-copyright analysis
(highlight reels) strictly separate. Person/pose/gait/face = do not publish without an Art.9
explicit-consent basis. Get EU + UK jurisdiction-specific advice before redistributing numeric
streams at scale."
    }
  ],
  "provenanceFork": "PROVENANCE IS THE DECISIVE FORK ? the legal posture of an identical numeric
file flips entirely on where the source video came from. FORK A ? OWNED / OWNER-RECORDED (or
fully-permissioned/licensed-for-this-use) SOURCE: This is the clean path and the only one safe
to commit publicly. No third-party copyright in the source bars the extraction; no third-party
ToS governs the owner's own recording (with two caveats below); the EU/UK database-right
'obtained vs created' debate still applies in theory but there is no adverse rights-holder to
assert it against the owner's own footage. The analysis collapses to: (1) a TRADE-SECRET /
competitive-moat CHOICE (US framing: 18 USC 1839 DTSA requires secrecy measures + value-from-
secrecy; the OPERATIVE framing for this Bangalore owner is Indian breach-of-confidence /
Contract Act 1872 / IT Act s.72, NOT the DTSA) ? publishing the data forfeits whatever moat it
carried, which is the owner's strategic call to make, weighed against the stated
community/benchmark/paper-co-authorship aims; and (2) PRIVACY ? the owner's own footage still
contains identifiable people, so DPDP/GDPR and the minors rules below still bind, and
person/pose/gait/face streams are still off-limits. TWO CAVEATS that narrow Fork A: (a) 'owner-
recorded' is clean ONLY for the owner's OWN non-ticketed/private sessions; footage the owner
SHOT AT a third party's event (tournament, club match) carries live exposure ? a ticket is a
contract of admission that routinely bars recording/redistribution, and
venue/organiser/broadcaster rights (e.g., BWF/league terms, Rome Convention rebroadcast rights)
attach independently of any platform ToS; (b) any third-party-collected footage in the corpus
(the project's golden set may mix sources) taints those records and pushes them into Fork B.
```

FORK B ? THIRD-PARTY-SOURCED (broadcast / YouTube / scraped / downloaded) SOURCE: Derived data from this provenance SHOULD NOT be committed to a public repo. Even though the numbers are uncopyrightable facts, (1) the EXTRACTION act needs a license or a now-CONTESTED US fair-use defense, and unlawful acquisition (e.g., scraping against ToS, downloading from a platform that bars it) independently sinks the defense (Bartz v. Anthropic); (2) the source's ToS (YouTube et al.) typically bars automated extraction AND redistribution, and committing the derived dataset publicly IS redistribution ? the highest-risk act, remediable by injunction + forced deletion of the committed data (hiQ); (3) re-utilisation/extraction of a substantial part may infringe the EU/UK sui generis database right if the source ties back to a protected feed. PRACTICAL RULE FOR THE REPO: gate the committed corpus to FORK A only. Tag every golden-set record with a verified provenance flag at ingestion; only owned/permissioned-source, non-event-ticketed, minors-excluded, person/pose/gait-excluded records are eligible for the public commit. This maps directly onto the existing split: eval_baselines/ (aggregate JSON snapshots) is already committed precedent and is provenance-agnostic if it carries no per-source attribution; *_golden_features.json are currently kept private (docs/GOLDEN_DATA_SHARING.md) precisely because they derive from a 'Proprietary corpus' ? the right move is to keep Fork-B-tainted features private and publish only a Fork-A, de-attributed subset.",

  "privacyVerdict": "PUBLISH/NO-PUBLISH BY STREAM (the project's guardrails already point the right way; this hardens them). PUBLISH-OK (lowest risk): (1) SHUTTLE (x,y) coordinates + Visibility ? describe an inanimate object, NOT personal data on their own (GDPR Art.4(1); WP136 content/purpose/result test) ? PROVIDED they are NOT joined to anything that singles out a named player and NOT published as an identifiable named match (the surrounding personal-data obligations survive even when the coordinate column alone is object-data). (2) Abstract RALLY start/end time-INTERVALS and ground-truth intervals stripped of match/player identity ? facts about event timing. (3) Non-biometric AUDIO-ONSET, COURT-BOUNDS, and aggregate PERSON COUNT/feature columns used ONLY for rally detection and DETACHED from any identity ? ordinary low-risk data with normal hygiene; NOT Art.9 because the purpose is detect/categorize, not uniquely identify (EDPB Guidelines 3/2019). CONDITIONAL ? only with an Art.9 explicit-consent basis (and effectively never for this v1): PERSON bounding-box coordinates that persistently single out and follow a specific player across rallies ? the purpose flips to unique identification and Art.9 attaches; this architecture is PRONE to that flip, so treat per-player tracking as a tripwire. DO NOT PUBLISH (high risk, exclude from any committed stream): (4) POSE keypoints / SKELETON / GAIT signatures ? gait is a behavioural biometric that re-identifies people even with faces blurred, and pose 'anonymization' provably leaves gait identity intact (Weiss et al., Sensors 2025, 80?95% gait-recognition on anonymized data); raw pose/gait tracks are NOT reliably anonymous, are personal data when usable to single out a player, and become Art.9 special-category once used to uniquely identify ? this matches the project's existing 'gait/biometrics strictly FUTURE (GDPR Art.9)' guardrail. (5) Any FACE CROP or identifiable IMAGERY ? identifying personal data under GDPR and DPDP, and barred by YouTube ToS's face-harvesting clause; do not commit imagery at all. (6) Anything that joins a NAMED PLAYER to a gait/pose signature. MINORS FLIP EVERYTHING: under DPDP a 'child' is anyone under 18; processing ANY child's data needs VERIFIABLE parental consent (Rule 10 ? real ID/age check, not a checkbox), must not cause detrimental effect, and s.9(3) PROHIBITS behavioural monitoring/tracking of children (Rule 12/Fourth-Schedule exemptions do not cover this project, so the ban applies). A persistent per-player pose/gait signature is, conservatively, prohibited monitoring. CLEANEST PATH for any footage containing minors: EXCLUDE minors entirely from any persisted derived stream beyond inanimate shuttle/interval data ? consistent with the existing 'EXCLUDE MINORS from the training pool' guardrail ? or obtain Rule-10 verified parental consent and keep only de-identified object/coordinate data. NON-ATTRIBUTABILITY ARCHITECTURE (the 'secret file' question): Non-attributability is a property of the (data, holder, environment) triple, not of the timeseries. A per-video trajectory/feature stream is effectively a FINGERPRINT (de Montjoye: 4 points re-identify 95% of 1.5M people; k-anonymity collapses on rich quasi-identifiers), so a deterministic detector re-run on a candidate video can correlate-match it to its source ? BUT only if a correlatable reference EXISTS. PUBLIC source: a permanent cheap reference exists, re-linkage is 'reasonably likely', so a raw public-sourced stream should be treated as ATTRIBUTABLE; only heavy aggregation, perturbation/differential privacy, synthetic data, or a non-public enclave (NIST SP 800-188) breaks correlatability. PRIVATE/owner-recorded-and-unpublished source: there is no external reference to correlate against, so source-linkage by an EXTERNAL adversary is not 'reasonably likely' ? BUT it is WRONG to call this 'anonymous by default' (EDPS v. SRB, CJEU C-413/23 P, 4 Sept 2025): to the holder who can re-run the detector, it REMAINS personal data; it is merely RELATIVELY non-attributable to outsiders, and that protection is conditional, revocable (publish or leak the source and the link snaps back), and forward-looking (regulators say assume the data environment grows). HARD ENGINEERING IMPLICATION: the committed fixtures must NOT carry the source filename, match metadata, or the video_id (which is the MD5 of the source file ? using it as a public key would

FINGERPRINT/confirm the source the instant anyone holds the file). Use an opaque, random, non-reversible surrogate id per record. Strip the 'name' field in *_golden_features.json. The lead's established constraint is correct: a contributor who RUNS the tests can necessarily read the plaintext fixtures and any key on their machine (DRM/analog-hole limit), so secrecy-from-the-contributor is impossible; the achievable goal is NON-ATTRIBUTABILITY + uselessness-in-isolation, with encryption useful ONLY as a barrier against the no-key public, never against a running contributor.",
  "conditionsForSafePublicCommit": [
    "PROVENANCE GATE: commit ONLY derived data from owner-recorded / fully-permissioned source video (Fork A). Tag every golden-set record with a verified provenance flag at ingestion; exclude any third-party-sourced (broadcast/YouTube/scraped/downloaded) record from the public commit. For owner-SHOT footage of a third party's event, first clear the admission ticket / venue / organiser / broadcaster terms ? only owner's own non-ticketed sessions are unconditionally clean.",
    "SOURCE-LICENCE/ToS CHECK: for every source that touched the committed corpus, confirm in writing that its licence/ToS does NOT bar automated extraction OR redistribution of derived data. Do not rely on US fair use for the extraction step ? it is now contested (Thomson Reuters v. Ross; USCO May-2025) and unlawful acquisition independently defeats it (Bartz v. Anthropic). Redistribution (the public commit) is the single highest-risk act.",
    "MINORS EXCLUSION: exclude all footage containing minors (under 18, per DPDP s.2(f)) from any persisted/committed derived stream beyond inanimate shuttle/interval data ? unless Rule-10 verifiable parental consent is held AND only de-identified object/coordinate data is kept. s.9(3)'s behavioural-monitoring ban means a persistent per-player pose/gait signature on a minor is prohibited.",
    "BIOMETRIC EXCLUSION: do NOT publish pose keypoints, skeleton tracks, gait signatures, face crops, identifiable imagery, or any named-player-to-movement-signature mapping. Restrict the committed feature columns to shuttle (x,y)+Visibility, abstract rally/GT time-intervals, and non-biometric audio/court/aggregate-person features used only for rally detection and detached from identity.",
    "DE-ATTRIBUTION: strip every source-identifying field from the committed fixtures ? source filename, match/event/player metadata, capture dates, and especially the video_id (MD5 of the source file). Replace with an opaque, random, non-reversible surrogate record id. The 'name' field in *_golden_features.json must be removed/aliased.",
    "NON-ATTRIBUTABILITY POSTURE: because per-video trajectories are fingerprints correlatable to a source video, achieve non-attributability by keeping the SOURCE private (Fork A, unpublished) ? which makes external source-linkage not 'reasonably likely' ? and/or by coarsening/aggregating/perturbing where feasible. Document the (data, holder, environment) assessment and re-test it as the public-video environment grows (ICO/OVIC forward-looking duty). Treat the streams as personal data IN THE OWNER'S OWN HANDS regardless (EDPS v. SRB).",
    "USELESS-IN-ISOLATION + ENCRYPTION-AS-PUBLIC-BARRIER ONLY: keep the committed fixtures useless without the regression harness + schema (abstract intervals + opaque ids, no imagery, no source pointer). Use encryption only as a barrier against the no-key public; do not represent it as protecting against a contributor who runs the tests (they can read plaintext + key locally ? the lead's established DRM/analog-hole constraint).",
    "PRECEDENT-CONSISTENT PLACEMENT: publish via the already-committed eval_baselines/ pattern (aggregate, provenance-agnostic JSON snapshots) for headline regression baselines; keep Fork-B-tainted or biometric-bearing *_golden_features.json private (per docs/GOLDEN_DATA_SHARING.md) and publish only a Fork-A, de-attributed, minors-excluded, biometric-free subset of features needed to run the suite.",
    "DPDP HYGIENE: ensure any person-derived numeric column that could single out a player is genuinely identity-detached; do not rely on the DPDP s.3(c)(ii) 'publicly available data' carve-out for non-anonymized personal data (it is contested). Maintain purpose-limitation: features persisted for rally detection must not be repurposed for player identification.",
    "COUNSEL SIGN-OFF: obtain retained India counsel (DPDP s.9/Rules, IT Act s.43/s.66, breach-of-confidence) AND EU/UK counsel (sui generis database right 'obtained vs created', CV-Online harm test, Ryanair contract) review of the final committed subset and the source-provenance log BEFORE the first public commit. This memo is general information, not advice."
  ],
  "redFlags": [
    "OVERCONFIDENCE TRAP #1 ? 'we created the data, so the EU/UK database right cannot apply.' Football Dataco v. Sportradar held live match-event data was OBTAINED (pre-existing on-court facts the recorder did not control), exactly analogous to a recorded badminton rally/shuttle position. A rights-holder can credibly argue the sui generis right subsists in our self-derived streams. Do not bank on the 'created' characterisation.",
    "OVERCONFIDENCE TRAP #2 ? 'analytic extraction is strongly favorable fair use (US).' Thomson

Reuters v. Ross (2025) REJECTED the 'copy an expressive work to reach unprotected facts' theory and found Factor-4 harm in a derivative data market; the USCO (May 2025) rejected 'non-expressive = inherently transformative.' If the dataset competes with a rights-holder's data/analytics market, fair use may fail. The extraction step is litigable, not settled.",
    "OVERCONFIDENCE TRAP #3 ? 'owner's own recordings have no contract problem.' False for footage shot at a third party's event: the admission ticket is a contract barring recording/redistribution, and venue/organiser/broadcaster rights (BWF/league/Rome Convention) attach independently. Only the owner's own non-ticketed sessions are unconditionally clean.",
    "OVERCONFIDENCE TRAP #4 ? 'private/unpublished source = anonymous by default.' Per EDPS v. SRB (CJEU 2025) the stream REMAINS personal data to the holder who can re-run the detector; it is only RELATIVELY non-attributable to an external adversary lacking the source, and that is conditional, revocable, and forward-looking. Misclassifying it as 'anonymous' is the exact GDPR error the controller's-perspective holding exists to prevent.",
    "FINGERPRINTING / video_id leak: the project's video_id is the MD5 of the source file. Committing it (or any source filename/match metadata) as a public key would let anyone holding the candidate video CONFIRM the source instantly ? defeating non-attributability. Must be stripped and replaced with an opaque surrogate.",
    "REDISTRIBUTION REMEDY is not nominal damages: the operative relief in scraping/ToS disputes is INJUNCTION + FORCED DELETION of the derived dataset (hiQ ordered destruction of all scraped-derived code/data). A public commit that breaches a source ToS can be ordered taken down and deleted ? a real, not theoretical, downside.",
    "POSE/GAIT mislabeled low-risk: skeleton 'anonymization' leaves gait identity intact (80?95% re-ID), so any pose/gait stream is NOT reliably anonymous and is Art.9-adjacent. Excluding it is not optional polish ? it is a guardrail.",
    "MINORS + behavioural monitoring: DPDP s.9(3) bars behavioural monitoring of under-18s and the Rule 12/Fourth-Schedule exemptions do not cover this project. A persistent per-player pose/gait signature on a minor is, conservatively, a prohibited act with criminal-adjacent regulatory exposure ? exclude minors from any persisted stream beyond inanimate shuttle/interval data.",
    "MIXED-PROVENANCE corpus: the golden set may blend owned and third-party footage. A single Fork-B-tainted record contaminates the public commit. Provenance must be verified per-record, not assumed corpus-wide.",
    "WRONG-LAW trade-secret framing: analysing the owner's moat under US 18 USC 1839 (DTSA) is a jurisdictional error ? the Bangalore owner's protection runs through Indian breach-of-confidence / Contract Act 1872 / IT Act s.72. The publish-forfeits-the-moat intuition survives, but the operative doctrine is different."
  ],
  "counselQuestions": [
    "EU/UK sui generis database right: on the Football Dataco v. Sportradar 'obtained vs created' reasoning, does the right subsist in our self-derived shuttle-coordinate / rally-event streams, and if so does redistributing a de-attributed regression subset 'extract/re-utilise a substantial part' and cause CV-Online-Latvia-type harm to any rights-holder's feed? (EU AND UK, given likely English-language market.)",
    "US fair use for the EXTRACTION step, post-Thomson Reuters v. Ross and the USCO May-2025 report: is transient analytic copying of a source video to compute rally/coordinate features defensible where the resulting dataset could be seen to compete in a 'data/analytics' market ? and does it matter that we only ever publish numbers, never the footage?",
    "India: are anti-scraping/ToS terms enforceable in Indian courts, and what is the realistic IT Act s.43 (civil) / s.66 (criminal) exposure for ingesting third-party footage to derive features ? and does the pending ANI v. OpenAI judgment (reserved, ~April 2026) change the fair-dealing/TDM analysis?",
    "India DPDP s.9 + Rules 10/12: for a corpus that may contain players under 18, what verifiable-parental-consent and exemption analysis applies, and does a persistent per-player pose/gait (or even per-player rally-tracking) feature constitute prohibited 'behavioural monitoring/tracking' of a child under s.9(3)?",
    "Owner-shot event footage: do the admission tickets / venue / organiser (e.g., BWF, local league) / broadcaster terms for the specific tournaments and clubs in our corpus bar recording, derivation, or redistribution of derived data ? and which records must therefore be excluded from the public commit?",
    "Personal-data classification under GDPR + DPDP for the SPECIFIC committed feature columns: at what point do identity-detached person/court/coordinate features 'single out' a player (especially in a small known squad/roster), and does our de-attribution + opaque-surrogate-id scheme suffice to keep the published subset outside personal-data obligations for downstream recipients (per EDPS v. SRB)?",
    "Trade-secret / breach-of-confidence (Indian law): given commercialization and possible

```
paper co-authorship, what is the moat trade-off of publishing the Fork-A subset, and are there
confidentiality/Contract-Act 1872 / IT Act s.72 steps to take before forfeiting secrecy on any
owner-recorded derived data?",
    "Source-licence audit: for each source category in the corpus (owner-recorded, permissioned
partner, downloaded/scraped), confirm the licence/ToS expressly permits BOTH automated
extraction AND public redistribution of derived numeric data ? and where silent or prohibitive,
which records are ineligible.",
    "Encryption + 'secret file' design: does adding encryption that a running contributor can
necessarily decrypt create any misrepresentation/false-security or DPDP 'reasonable security
safeguards' compliance question, and is non-attributability-via-private-source a defensible
posture to assert to regulators given its conditional/revocable nature?",
    "Cross-border data transfer: does committing any record that is (or may be argued to be)
personal data to a global public GitHub repo implicate DPDP cross-border-transfer rules or GDPR
international-transfer obligations, independent of the copyright/contract analysis?"
  ],
  "citations": [
    {
      "name": "Feist Publications, Inc. v. Rural Telephone Service Co., 499 U.S. 340 (1991) ?
Justia",
      "url": "https://supreme.justia.com/cases/federal/us/499/340/"
    },
    {
      "name": "National Basketball Ass'n v. Motorola, Inc., 105 F.3d 841 (2d Cir. 1997) ?
Wikipedia case summary",
      "url": "https://en.wikipedia.org/wiki/National_Basketball_Ass%27n_v._Motorola,_Inc."
    },
    {
      "name": "17 U.S.C. § 102 ? BitLaw / U.S. Code",
      "url": "https://www.bitlaw.com/source/17usc/102.html"
    },
    {
      "name": "17 U.S.C. § 101 Definitions ? Legal Information Institute (Cornell)",
      "url": "https://www.law.cornell.edu/uscode/text/17/101"
    },
    {
      "name": "Sega Enterprises Ltd. v. Accolade, Inc., 977 F.2d 1510 (9th Cir. 1992) ? BitLaw",
      "url": "https://www.bitlaw.com/source/cases/copyright/Sega-Accolade.html"
    },
    {
      "name": "Authors Guild v. Google, Inc., 804 F.3d 202 (2d Cir. 2015) ? Justia",
      "url": "https://law.justia.com/cases/federal/appellate-
courts/ca2/13-4829/13-4829-2015-10-16.html"
    },
    {
      "name": "Google LLC v. Oracle Am., Inc., 141 S. Ct. 1183 (2021) ? U.S. Copyright Office
Fair Use Index summary",
      "url": "https://www.copyright.gov/fair-use/summaries/google-llc-oracle-am-inc-2021.pdf"
    },
    {
      "name": "Thomson Reuters Enterprise Centre GmbH v. Ross Intelligence Inc. (D. Del. 2025) ?
full opinion (BitLaw)",
      "url": "https://www.bitlaw.com/source/cases/copyright/Thomson-Reuters-Ross.html"
    },
    {
      "name": "Perkins Coie ? Fair Use Defense Failed in Thomson Reuters v. Ross",
      "url": "https://perkinscoie.com/insights/update/fair-use-defense-failed-thomson-reuters-v-
ross-jury-still-out-generative-ai"
    },
    {
      "name": "ArentFox Schiff ? Landmark Ruling on AI Copyright: Bartz v. Anthropic",
      "url": "https://www.afslaw.com/perspectives/alerts/landmark-ruling-ai-copyright-fair-use-
vs-infringement-bartz-v-anthropic"
    },
    {
      "name": "Authors Guild ? Bartz v. Anthropic Settlement (~$1.5B; piracy-based liability)",
```

```
      "url": "https://authorsguild.org/advocacy/artificial-intelligence/what-authors-need-to-
know-about-the-anthropic-settlement/"
    },
    {
      "name": "Mayer Brown ? U.S. Copyright Office (May 2025, Part 3) on AI training and fair
use",
      "url": "https://www.mayerbrown.com/en/insights/publications/2025/05/united-states-
copyright-office-weighs-in-on-fair-use-defense-for-generative-ai-training"
    },
    {
      "name": "ProCD, Inc. v. Zeidenberg, 86 F.3d 1447 (7th Cir. 1996) ? Wikipedia",
      "url": "https://en.wikipedia.org/wiki/ProCD,_Inc._v._Zeidenberg"
    },
    {
      "name": "EUR-Lex ? Case C-203/02 British Horseracing Board v William Hill (9 Nov 2004)",
      "url": "https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex%3A62002CJ0203"
    },
    {
      "name": "Wolters Kluwer / Kluwer Copyright Blog ? Database Directive sports-data cases",
      "url": "https://legalblogs.wolterskluwer.com/copyright-blog/databases-sui-generis-
protection-and-copyright-protection/"
    },
    {
      "name": "EUR-Lex ? Case C-604/10 Football Dataco v Yahoo (1 Mar 2012)",
      "url": "https://eur-lex.europa.eu/legal-content/EN/ALL/?uri=CELEX:62010CJ0604"
    },
    {
      "name": "ipcuria ? Case C-173/11 Football Dataco v Sportradar (18 Oct 2012)",
      "url": "https://ipcuria.eu/case?reference=C-173/11"
    },
    {
      "name": "ipcuria ? Case C-762/19 CV-Online Latvia v Melons (3 Jun 2021)",
      "url": "https://ipcuria.eu/case?reference=C-762%2F19"
    },
    {
      "name": "Bird & Bird ? CV-Online Latvia: CJEU complicates enforcement of database rights",
      "url": "https://www.twobirds.com/en/insights/2021/uk/cv-online-latvia-cjeu-complicates-
the-enforcement-of-database-rights"
    },
    {
      "name": "Simkins ? Database rights back on the sport radar (Floyd J: live match events
'obtained')",
      "url": "https://www.simkins.com/news/database-rights-back-on-the-sport-radar"
    },
    {
      "name": "LexisNexis UK ? Legal protection of databases in the UK (post-Brexit assimilated
sui generis right)",
      "url": "https://www.lexisnexis.com/en-gb/legal/guidance/legal-protection-of-databases-in-
the-uk"
    },
    {
      "name": "EUR-Lex ? Case C-30/14 Ryanair v PR Aviation (15 Jan 2015)",
      "url": "https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:62014CJ0030"
    },
    {
      "name": "EBC v D.B. Modak (WIPO Lex / Supreme Court of India)",
      "url": "https://www.wipo.int/wipolex/en/judgments/details/2840"
    },
    {
      "name": "Ikigai Law ? Legality of data scraping in India (OLX v Padawan; IT Act s.43; ToS;
database copyright)",
      "url": "https://www.ikigailaw.com/article/263/legality-of-data-scraping-in-india"
    },
    {
      "name": "IAPP ? Scraping public data in India: innovation enabler or privacy threat? (Govt
```

```
s.43 position; DPDP s.3(c)(ii))",
      "url": "https://iapp.org/news/a/scraping-public-data-in-india-innovation-enabler-or-
privacy-threat-"
    },
    {
      "name": "CaseMine ? Burlington Home Shopping v Rajnish Chibber (database as literary
work)",
      "url": "https://www.casemine.com/commentary/in/recognition-of-database-compilation-as-a-
literary-work:-analysis-of-burlington-home-shopping-pvt-ltd.-v.-rajnish-chibber-&-anr./view"
    },
    {
      "name": "CaseMine ? V. Govindan v E.M. Gopalakrishna Kone (compilation copyright)",
      "url": "https://www.casemine.com/commentary/in/establishing-copyright-infringement-in-
dictionary-compilations:-v.-govindan-v.-e.m-gopalakrishna-kone/view"
    },
    {
      "name": "S.S. Rana & Co. ? Data scraping issues in India (IT Act s.43/s.66 criminal
exposure)",
      "url": "https://ssrana.in/articles/data-scraping-issues-india/"
    },
    {
      "name": "The New Publishing Standard ? Delhi HC reserves judgment in ANI v OpenAI (April
2026)",
      "url": "https://thenewpublishingstandard.com/2026/04/01/ani-vs-openai-delhi-high-court-
judgment-ai-copyright-india/"
    },
    {
      "name": "DPDPA s.9 ? children's data: verifiable consent + behavioural-tracking/targeted-
ads prohibition",
      "url": "https://www.dpdpa.com/dpdpa2023/chapter-2/section9.html"
    },
    {
      "name": "DPDP Rules 2025, Rule 10 ? verifiable parental consent",
      "url": "https://www.dpdpa.com/dpdparules/rule10.html"
    },
    {
      "name": "DPDP Rules 2025, Rule 12 ? exemptions from s.9(1)/s.9(3) for Fourth Schedule
classes/purposes",
      "url": "https://www.dpdpa.com/dpdparules/rule12.html"
    },
    {
      "name": "Medianama ? New Data Rules Allow Child Tracking for Safety Without Consent (Rule
12 carve-out)",
      "url": "https://www.medianama.com/2025/11/223-dpdp-rules-real-time-child-tracking-without-
consent/"
    },
    {
      "name": "CyberPeace ? Prohibition of behavioural tracking and targeted advertising for
children under DPDP s.9(3)",
      "url": "https://cyberpeace.org/resources/blogs/prohibition-of-behavioral-tracking-and-
targeted-advertising-for-children-under-the-dpdp-act-2023"
    },
    {
      "name": "S.S. Rana & Co. ? Protecting trade secrets in India (Contract Act 1872, breach of
confidence, IT Act s.72)",
      "url": "https://ssrana.in/articles/protecting-trade-secrets-india/"
    },
    {
      "name": "K Singhania & Co. ? Regulation of biometric data under the DPDP Act (facial data
= personal data)",
      "url": "https://ksandk.com/data-protection-and-data-privacy/regulation-of-biometric-data-
under-the-dpdp-act/"
    },
    {
      "name": "Van Buren v. United States, No. 19-783 (U.S. June 3, 2021) ? slip opinion",
```

```
      "url": "https://www.supremecourt.gov/opinions/20pdf/19-783_k53l.pdf"
    },
    {
      "name": "Zwillgen ? Van Buren: footnote 8 reserved whether contract/policy limits ground
CFAA liability",
      "url": "https://www.zwillgen.com/alternative-data/scotus-rules-cfaa-does-not-contain-
purpose-based-restrictions/"
    },
    {
      "name": "Privacy World (Squire Patton Boggs) ? hiQ v. LinkedIn breach-of-contract summary
judgment",
      "url": "https://www.privacyworld.blog/2022/11/federal-court-rules-in-favor-of-linkedins-
breach-of-contract-claim-after-six-years-of-cfaa-data-scraping-litigation/"
    },
    {
      "name": "Privacy World ? hiQ v. LinkedIn $500k consent judgment, injunction and
destruction of scraped-derived data",
      "url": "https://www.privacyworld.blog/2022/12/linkedins-data-scraping-battle-with-hiq-
labs-ends-with-proposed-judgment/"
    },
    {
      "name": "Eric Goldman / Quinn Emanuel ? Meta v. Bright Data (N.D. Cal. Jan. 23, 2024):
logged-off public scraping not barred",
      "url": "https://www.quinnemanuel.com/the-firm/news-events/client-alert-meta-v-bright-data-
significant-decision-for-web-scraping-industry/"
    },
    {
      "name": "Crowell & Moring ? browsewrap vs clickwrap enforceability (notice + assent)",
      "url": "https://www.crowell.com/en/insights/client-alerts/recent-court-rulings-provide-
warnings-on-the-use-of-browsewrap-agreements"
    },
    {
      "name": "Hunton ? sign-in wrap enforceability: courts deny absent conspicuous notice /
unambiguous assent",
      "url": "https://www.hunton.com/privacy-and-information-security-law/its-a-wrap-the-latest-
from-the-ninth-circuit-on-sign-in-wrap-agreements"
    },
    {
      "name": "YouTube Terms of Service ? Permissions and Restrictions (automated access;
harvest faces; redistribution)",
      "url": "https://www.youtube.com/static?template=terms"
    },
    {
      "name": "18 U.S.C. § 1839 ? Trade Secret Definitions (Cornell LII)",
      "url": "https://www.law.cornell.edu/uscode/text/18/1839"
    },
    {
      "name": "Gowling WLG ? Trade secrets in professional sports",
      "url": "https://gowlingwlg.com/en/insights-resources/articles/2023/trade-secrets-in-
professional-sports"
    },
    {
      "name": "Lexology ? In-venue filming of live sporting events: ticket as contract;
venue/broadcaster rights",
      "url": "https://www.lexology.com/library/detail.aspx?g=09fe6573-daf6-4fff-
bcf0-b9c6ccf41792"
    },
    {
      "name": "GDPR Recital 26 (official text, gdpr-info.eu)",
      "url": "https://gdpr-info.eu/recitals/no-26/"
    },
    {
      "name": "gdpr-info.eu ? Personal Data (Art.4(1)); WP136 'relates to' test",
      "url": "https://gdpr-info.eu/issues/personal-data/"
    },
```

```
    {
      "name": "gdpr-info.eu ? Art.9 GDPR (special-category biometric data); ICO biometric
guidance",
      "url": "https://gdpr-info.eu/art-9-gdpr/"
    },
    {
      "name": "gdpr-info.eu ? Art.8 GDPR (children's consent)",
      "url": "https://gdpr-info.eu/art-8-gdpr/"
    },
    {
      "name": "Fox Rothschild ? EDPB Guidelines 3/2019 on video devices (Art.9 only on unique
identification)",
      "url": "https://www.foxrothschild.com/publications/video-surveillance-under-gdpr-edpb-
issues-final-guidance"
    },
    {
      "name": "Weiss et al., Sensors 2025 (PMC) ? Privacy Beyond the Face: gait identity
survives pose anonymization",
      "url": "https://pmc.ncbi.nlm.nih.gov/articles/PMC12788357/"
    },
    {
      "name": "Bird & Bird ? CJEU C-582/14 Breyer: relative approach to identifiability",
      "url": "https://www.twobirds.com/en/insights/2016/global/cjeu-decision-on-dynamic-ip-
addresses-touches-fundamental-dp-law-questions"
    },
    {
      "name": "Article 29 Working Party Opinion 05/2014 (WP216) on Anonymisation Techniques",
      "url": "https://ec.europa.eu/justice/article-29/documentation/opinion-
recommendation/files/2014/wp216_en.pdf"
    },
    {
      "name": "Hunton ? EDPB Guidelines 01/2025 on pseudonymisation (context-dependent
identifiability)",
      "url": "https://www.hunton.com/insights/publications/edpb-advises-on-pseudonymisation-for-
gdpr-compliance"
    },
    {
      "name": "Clifford Chance ? Pseudonymized data after EDPS v SRB (CJEU C-413/23 P, 4 Sept
2025)",
      "url": "https://www.cliffordchance.com/insights/resources/blogs/talking-
tech/en/articles/2025/09/pseudonymized-data-after-edps-v-srb.html"
    },
    {
      "name": "Skadden ? EU Court Clarifies When Pseudonymised Data Is Not Personal Data (EDPS v
SRB)",
      "url": "https://www.skadden.com/insights/publications/2025/11/in-a-landmark-decision-eu-
court-clarifies"
    },
    {
      "name": "ICO ? How do we ensure anonymisation is effective? (motivated-intruder test)",
      "url": "https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-
sharing/anonymisation/how-do-we-ensure-anonymisation-is-effective/"
    },
    {
      "name": "NIST CSRC ? SP 800-188 De-Identifying Government Datasets (release-model
framing)",
      "url": "https://csrc.nist.gov/News/2023/nist-publishes-sp-800-188"
    },
    {
      "name": "Sweeney, k-Anonymity: A Model for Protecting Privacy (EPIC mirror)",
      "url": "https://epic.org/wp-content/uploads/privacy/reidentification/Sweeney_Article.pdf"
    },
    {
      "name": "de Montjoye et al., Unique in the Crowd (Scientific Reports 2013) ? 4 points ->
95%",
```

```
      "url": "https://dspace.mit.edu/bitstream/handle/1721.1/88233/de%20Montjoye-2013-
Unique%20in%20the%20crowd.pdf?sequence=1&isAllowed=y"
    },
    {
      "name": "OVIC ? The Limitations of De-Identification (data environment / external linkage;
forward-looking)",
      "url": "https://ovic.vic.gov.au/privacy/resources-for-organisations/the-limitations-of-de-
identification-protecting-unit-record-level-personal-information/"
    },
    {
      "name": "Trajectory-Based Spatiotemporal Entity Linking (arXiv 2010.01516)",
      "url": "https://arxiv.org/pdf/2010.01516"
    }
  ]
}

GROUNDING:
CODEBASE FACTS (verified by the lead before this run):
- eval_baselines/ is a TRACKED (committed) dir of JSON baseline snapshots
(regression_baseline.json, heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json,
experiment_leaderboard.json). Precedent: committed structured regression files already live in
git.
- backend/eval/regression.py loads a committed baseline, scores DB-stored predictions vs ground-
truth intervals, flags P/R drop beyond tolerance, with incommensurability guards
(IoU/detector/GT-hash). Predictions come from the DB (needs a pipeline run on the video) -> NOT
video-free on the prediction side yet.
- backend/eval/fusion_golden.py extracts *_golden_features.json from VIDEO+trajectory CSV+labels
(needs GPU/WSL). backend/eval/fusion_compare.py & backend/eval/ablation.py RE-SCORE those JSON
features on pure CPU with NO video (LOVO deltaF1, bootstrap CI, sign test). This is the EXISTING
VIDEO-FREE seam.
- docs/GOLDEN_DATA_SHARING.md (#175): *_golden_features.json are deliberately kept PRIVATE on
OneDrive, OUT of git, because they derive from a 'Proprietary corpus.' Existing skipif litmus
tests (tests/test_ablation.py::test_litmus_reproduces_gate_a, tests/test_fusion_compare.py) run
ONLY if those features are present in output/ -> currently skipped everywhere.
- Data contracts: trajectory CSV columns (Frame,Visibility,X,Y); candidate/final rally segments
= (start,end) time intervals; multi-signal fusion uses shuttle + person + audio + court-bounds
feature columns (weak signals retained, default-OFF). video_id = MD5 of the source file
(backend/utils/hashing.py) ? i.e. using it as a public key would FINGERPRINT the source file.
- Guardrails: commercial-clean licensing MIT/Apache/BSD only; paid LLMs never a training source;
EXCLUDE MINORS from the training pool; gait/biometrics strictly FUTURE (GDPR Art 9). Owner is
Bangalore/India-based; aims at commercialization and possibly co-authoring a research paper.
- A SEPARATE workflow is concurrently designing the general video-free fixture mechanics. THIS
effort focuses on (a) the LEGAL/IP/privacy status of committing DERIVED structured data
PUBLICLY, and (b) a non-attributability + encryption ('secret file') architecture.
GOAL: let an external contributor on another clone run the rally-segmentation regression suite
from COMMITTED structured fixtures (no video present), such that the fixtures (i) cannot be
attributed to a specific source video/match/person and (ii) are useless to a public user in
isolation.
KEY CONSTRAINT ALREADY ESTABLISHED BY THE LEAD: a contributor who RUNS the tests necessarily can
read the plaintext fixtures + any key on their machine (DRM/analog-hole limit). So secrecy-from-
contributor is impossible; the achievable goal is NON-ATTRIBUTABILITY + uselessness-in-
isolation, plus encryption only as defense against the NO-KEY public.
```

2. assistant (2026-06-16T03:50:55.290Z)

I'll start by reading the established conventions and verifying the codebase facts before architecting the unified design. Let me pull the latest and read the key infrastructure.

3. user (2026-06-16T03:50:58.774Z)

```
Already up to date.
---STATUS---
960c00d docs: rally-quality research + eval-framework roadmap (foundation doc) (#184)
b6a39c4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)
c7df281 perf(wasb): stream video frames to the detector instead of extracting ~46k PNGs (#178)
```

4. user (2026-06-16T03:50:58.938Z)

```
eval_baselines\heuristic_lovo_n6.json
eval_baselines\gemini_wrapper_2026-06-10.json
eval_baselines\experiment_leaderboard.json
```

5. user (2026-06-16T03:50:58.945Z)

```
backend\eval\__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\eval\gemini_refine.py
backend\eval\export_wasb_segments.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\eval\segmentation_metrics.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\experiment.py
backend\eval\batch.py
backend\eval\fusion_golden.py
backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
backend\eval\ablation.py
backend\eval\served_gate_a.py
backend\eval\distill_local.py
backend\eval\promotion.py
backend\eval\serve_contrast.py
backend\eval\windowing.py
backend\eval\splits.py
```

6. assistant (2026-06-16T03:51:02.113Z)

Let me read the core modules that the design references, to verify the contracts and seams.

7. user (2026-06-16T03:51:03.074Z)

```
1       """Pure temporal interval-matching metrics for rally evaluation.
2
3       No I/O, no DB, no global state ? every function here is deterministic and unit
4       tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5       start < end. We score a list of *predicted* rally intervals against a list of
6       *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7       """
8
9       from dataclasses import dataclass, field
10      from typing import List, Tuple, Optional
11
12      Interval = Tuple[float, float]
13
14
15      def interval_iou(a: Interval, b: Interval) -> float:
16          """Temporal intersection-over-union of two intervals.
```

```
17
18          Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19          overlap / union, both measured as durations on the timeline.
20          """
21          a_start, a_end = a
22          b_start, b_end = b
23          if a_end <= a_start or b_end <= b_start:
24              return 0.0
25          inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26          if inter <= 0.0:
27              return 0.0
28          union = (a_end - a_start) + (b_end - b_start) - inter
29          return inter / union if union > 0 else 0.0
30
31
32      @dataclass
33      class Match:
34          """One matched predicted?ground-truth pair."""
35          pred_index: int
36          gt_index: int
37          iou: float
38
39
40      @dataclass
41      class MatchResult:
42          """Outcome of matching predictions to ground truth at a given IoU threshold."""
43          matches: List[Match] = field(default_factory=list)
44          unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
45          unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)
46
47
48      def match_intervals(preds: List[Interval], gts: List[Interval],
49                          iou_threshold: float = 0.5) -> MatchResult:
50          """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52          Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53          are claimed highest-IoU first, and each prediction/GT can be used once. This
54          keeps a single prediction from being credited for two ground-truth rallies
55          (and vice versa).
56          """
57          candidates = []
58          for pi, p in enumerate(preds):
59              for gi, g in enumerate(gts):
60                  iou = interval_iou(p, g)
61                  # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
62                  # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63                  if iou > 0.0 and iou >= iou_threshold:
64                      candidates.append((iou, pi, gi))
65          # Highest IoU first; ties broken by indices for determinism.
66          candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68          used_pred, used_gt = set(), set()
69          matches: List[Match] = []
70          for iou, pi, gi in candidates:
71              if pi in used_pred or gi in used_gt:
72                  continue
73              used_pred.add(pi)
74              used_gt.add(gi)
75              matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77          unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78          unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79          return MatchResult(matches=matches,
```

```
                            unmatched_pred_indices=unmatched_pred,
                            unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
        true_positives: int
        false_positives: int
        false_negatives: int
        precision: float
        recall: float
        f1: float
        mean_iou: float            # over matched pairs (0.0 if none)
        mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
        mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds

        def as_dict(self) -> dict:
            return self.__dict__.copy()


    def score(preds: List[Interval], gts: List[Interval],
              iou_threshold: float = 0.5,
              match_result: Optional[MatchResult] = None) -> Score:
        """Compute precision/recall/F1, mean matched IoU, and boundary errors.

        Pass a precomputed `match_result` to avoid re-matching (e.g. when also
        extracting the failure lists); otherwise matching is done here.
        """
        mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)

        tp = len(mr.matches)
        fp = len(mr.unmatched_pred_indices)
        fn = len(mr.unmatched_gt_indices)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0

        if mr.matches:
            mean_iou = sum(m.iou for m in mr.matches) / tp
            mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
            mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
        else:
            mean_iou = mean_start_err = mean_end_err = 0.0

        return Score(
            iou_threshold=iou_threshold,
            num_pred=len(preds), num_gt=len(gts),
            true_positives=tp, false_positives=fp, false_negatives=fn,
            precision=precision, recall=recall, f1=f1,
            mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
        )


    def pr_curve(preds: List[Interval], gts: List[Interval],
                 thresholds: Optional[List[float]] = None) -> List[Score]:
        """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
```

```
141
142         Lets you see strict-vs-lenient behaviour rather than a single operating point.
143         """
144         if thresholds is None:
145             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147
```

8. user (2026-06-16T03:51:04.034Z)

```
1       """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3       The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4       and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5       no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6       designed seam): the person features are extra columns the learned LogisticRegression
weights,
7       so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9       Run (no GPU):
10          python -m backend.eval.fusion_compare --features-dir output --record
11      """
12      from __future__ import annotations
13
14      import argparse
15      import glob
16      import json
17      import os
18      from typing import Any, Dict, List
19
20      from backend.eval import experiment
21      from backend.eval.eval_stats import paired_delta_ci
22      from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25      def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26          """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27          (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28          by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29          by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30          by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31          vids = [p["video"] for p in res["variant_a"]["per_video"]]
32          return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35      def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36          """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37          feature columns + golden gts). The harness derives labels from gts itself."""
38          videos: List[experiment.VideoCandidates] = []
39          for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40              with open(path, encoding="utf-8") as f:
41                  d = json.load(f)
42              cands = d["candidates"]
43              intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44              features: Dict[str, List[float]] = {}
45              for fn in SHUTTLE_FEATURE_NAMES:
46                  features[fn] = [float(c["shuttle"][fn]) for c in cands]
47              for fn in PERSON_FEATURE_NAMES:
```

```
                features[fn] = [float(c["person"][fn]) for c in cands]
            gts = [(float(a), float(b)) for a, b in d["gts"]]
            videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
                                                     features=features, gts=gts))
        return videos


    def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
                threshold: float = 0.5) -> Dict[str, Any]:
        shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
                                          threshold=threshold)
        fusion = experiment.Variant(name="shuttle_person_fusion",
                                    feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
                                    threshold=threshold)
        return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)


    def format_result(res: Dict[str, Any], iou: float) -> str:
        a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
        ds = delta_stats(res)
        st = ds["sign_test"]
        lines = [
            f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
            f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
            f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
            f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
            f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
            f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
            f"| CI clears 0: {ds['ci_excludes_zero']}",
            "  per-video ?F1:",
        ]
        for pv in res["per_video_delta"]:
            lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
        return "\n".join(lines)


    def main() -> None:
        ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
        ap.add_argument("--features-dir", default="output")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
        args = ap.parse_args()

        videos = load_videos(args.features_dir)
        if len(videos) < 2:
            raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
        res = compare(videos, iou=args.iou)
        print(format_result(res, args.iou))
        if args.record:
            row = experiment.record_experiment(res)
            print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']})")


    if __name__ == "__main__":
```

```
102         main()
103
```

9. user (2026-06-16T03:51:04.095Z)

```
1       """Regression runner ? verify the production tool against golden-set ground truth.
2
3       Implements the explicit ask: *"given a video, get a golden set created and verify
4       its precision and recall against the production tool."* This is deliberately a thin
5       wrapper over machinery that already exists:
6
7           ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8           predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
9           comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
10          baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12      A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13      baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14      later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15      independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16      """
17      import os
18      import json
19      import hashlib
20      import logging
21      from dataclasses import dataclass, asdict
22      from typing import Dict, List, Optional
23
24      from backend.infrastructure.database import Database
25      from backend.eval import metrics
26      from backend.eval.harness import get_predictions
27      from backend.annotations import annotation_registry
28
29      logger = logging.getLogger(__name__)
30
31      # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32      # actually load a committed baseline to compare against.
33      DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34      DEFAULT_TOLERANCE = 0.05
35
36
37      def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38          """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39          norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40          return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43      @dataclass
44      class RegressionResult:
45          video_id: str
46          stage: str
47          iou_threshold: float
48          precision: float
49          recall: float
50          f1: float
51          # Baseline values compared against (None when no baseline existed yet).
52          baseline_precision: Optional[float]
53          baseline_recall: Optional[float]
54          tolerance: float
55          regressed: bool
56          reasons: List[str]
57          # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58          # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
```

```
59          has_baseline: bool = False
60          gt_hash: Optional[str] = None
61
62          def as_dict(self) -> dict:
63              return asdict(self)
64
65
66      # ------------------------------------------------------------------ baseline store
67
68      def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69          """Load the per-video baseline snapshot file (returns {} if absent)."""
70          if not os.path.isfile(path):
71              return {}
72          with open(path) as f:
73              return json.load(f)
74
75
76      def _baseline_key(video_id: str, stage: str) -> str:
77          return f"{video_id}::{stage}"
78
79
80      def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                        f1: float, path: str = DEFAULT_BASELINE_PATH,
82                        iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                        gt_fingerprint: Optional[str] = None) -> None:
84          """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86          Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87          later run computed under different settings (or against different labels) is
detected
88          as incommensurable rather than silently compared.
89          """
90          baselines = load_baselines(path)
91          baselines[_baseline_key(video_id, stage)] = {
92              "video_id": video_id, "stage": stage,
93              "precision": precision, "recall": recall, "f1": f1,
94              "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95          }
96          os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97          with open(path, "w") as f:
98              json.dump(baselines, f, indent=2, sort_keys=True)
99          logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)
100
101
102     # ------------------------------------------------------------------ the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105                 stage: str = "final", iou_threshold: float = 0.5,
106                 detector: Optional[str] = None,
107                 tolerance: float = DEFAULT_TOLERANCE,
108                 baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
baseline.
110
111         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113         precision OR recall falls more than `tolerance` below the snapshotted baseline.
114         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115         caller can choose to snapshot the current numbers via `save_baseline`.
116         """
117         preds = get_predictions(db, video_id, stage, detector=detector)
118         s = metrics.score(preds, ground_truth, iou_threshold)
119         fp = gt_hash(ground_truth)
120
```

```
        baselines = load_baselines(baseline_path)
        b = baselines.get(_baseline_key(video_id, stage))

        reasons: List[str] = []
        regressed = False
        has_baseline = b is not None
        base_p = base_r = None
        if b is not None:
            base_p, base_r = b["precision"], b["recall"]
            # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
            # against different labels is not a valid reference ? flag it instead of
trusting it.
            b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
            if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
                regressed = True
                reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
            if b_det is not None and b_det != detector:
                regressed = True
                reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
            if b_hash is not None and b_hash != fp:
                regressed = True
                reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
            if s.precision < base_p - tolerance:
                regressed = True
                reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
            if s.recall < base_r - tolerance:
                regressed = True
                reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
        else:
            reasons.append("no baseline recorded for this video/stage ? nothing to compare")

        return RegressionResult(
            video_id=video_id, stage=stage, iou_threshold=iou_threshold,
            precision=s.precision, recall=s.recall, f1=s.f1,
            baseline_precision=base_p, baseline_recall=base_r,
            tolerance=tolerance, regressed=regressed, reasons=reasons,
            has_baseline=has_baseline, gt_hash=fp,
        )


    def regress_with_provider(db: Database, video_id: str, video_ref: str,
                              tier: str = "file", stage: str = "final",
                              iou_threshold: float = 0.5,
                              detector: Optional[str] = None,
                              tolerance: float = DEFAULT_TOLERANCE,
                              baseline_path: str = DEFAULT_BASELINE_PATH,
                              guideline: Optional[str] = None,
                              sport: str = "badminton",
                              provider_kwargs: Optional[dict] = None) -> RegressionResult:
        """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.

        `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
        once those providers land). `video_ref` is whatever that provider expects (a CSV
        path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
        predictions in the DB (the file MD5).
        """
        provider = annotation_registry.get(tier, **(provider_kwargs or {}))
```

```
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                        detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "".join(f"\n   - {why}" for why in r.reasons)
189
```

10. assistant (2026-06-16T03:51:10.600Z)

The core facts check out. Let me verify the remaining anchors: the feature-name constants, the golden-features schema producer, the existing skipif test pattern, and the committed baseline that the design flags as a live leak.

11. user (2026-06-16T03:51:11.439Z)

```
1       {
2         "name": "heuristic_velocity_windowing_LOVO",
3         "description": "The honest cross-video floor the Gen-0 learned model must beat:
calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
4         "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local
--manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
5         "mean_f1": 0.480,
6         "std": 0.242,
7         "per_video": {
8           "adarsh_avi_singles": 0.751,
9           "kushagra_singles": 0.157,
10          "largetest_doubles": 0.737,
11          "gbaaddy": 0.309,
12          "testlarge_short": 0.267,
13          "mahadevpura_singles": 0.657
14        }
15      }
16
```

12. user (2026-06-16T03:51:11.458Z)

```
1       """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3       A *systematic, repeatable* version of the one-off ablation
(``scratch/gate_ab_ablation.py``):
4       run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
per
5       signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
bootstrap
6       95% CI + paired sign test + the C4 over-segmentation ratio. See
``scratch/ABLATION_FRAMEWORK_PLAN.md``
7       (the design) and ``docs/ABLATION_LAYER.md`` (the shipped note).
8
9       **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
10      ``experiment.compare`` + ``eval_stats`` engine, exactly as ``compare`` was a driver over
11      ``metrics``/``calibration``. For each signal ``s`` the marginal contribution is:
12
13          ?(s) = score(full set S) ? score(S \\ {s})            # leave-one-out drop, ?F1
primary
14
```

```
15      operationally ``experiment.compare(videos, variant_a=without_s, variant_b=full)`` ? A is
the
16      ablated (smaller) variant, B is full, so the reported B?A delta IS ?(s), a per-video-
paired ?F1
17      with bootstrap CI + exact sign test, LOVO-honest for learned variants.
18
19      Two ablation granularities fall out of one inventory:
20
21        - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
22        - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
23          "is this old SIGNAL still relevant?" question at cue granularity).
24
25      A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26      a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
by
27      dropping a column). The runner handles both uniformly: ``without_s`` subtracts the
columns AND/OR
28      zeroes the gate knob from ``cfg.full``.
29
30      Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; §6 of the
plan):
31        - never a bare mean ? every row ships a CI + the exact sign test;
32        - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
sign test
33          agreeing** (the ``per_user_gate`` bar, in reverse);
34        - **structural signals are exempt from auto-demotion** ? a guard whose value is in
failure modes
35          the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36          flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37          but tags ``structural_protected``;
38        - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39          MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41      Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42      writes one ``<name>_golden_features.json`` per video). CLI::
43
44          python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45      """
46      from __future__ import annotations
47
48      import argparse
49      import glob
50      import json
51      import logging
52      import os
53      from dataclasses import dataclass, field
54      from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56      from backend.eval import experiment
57      from backend.eval.experiment import Variant, VideoCandidates
58
59      logger = logging.getLogger(__name__)
60
61      # --- The signal inventory (the things the runner ablates), grouped by producer.
---------
62      # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63      # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
```

```
64      # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
65      SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                          "active_ratio", "net_crossings")
67      PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
68                                         "in_court_ratio", "shuttle_player_colocation")
69      AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
70
71      # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
72      EARNS_KEEP = "earns_keep"               # CI clears 0 AND sign test agrees ? removing it
hurts
73      SUGGESTIVE = "suggestive"               # point estimate leans positive but CI spans 0
74      INERT = "inert"                         # wash this run (?0, CI spans 0) ? candidate,
not proof
75      STRUCTURAL_PROTECTED = "structural_protected"   # a guard; never auto-demoted on "no
lift"
76
77      #: Human-readable one-liners for each verdict (used in the PDF + table legend).
78      VERDICT_LABEL = {
79          EARNS_KEEP: "clears-bar",
80          SUGGESTIVE: "suggestive",
81          INERT: "inert",
82          STRUCTURAL_PROTECTED: "structural-protected",
83      }
84
85
86      @dataclass(frozen=True)
87      class Signal:
88          """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
89
90          - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
91          - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
whole group.
92          - ``columns`` ? the persisted feature columns this signal contributes (may be empty
for a
93            pure gate signal like Gate-A whose column is shared with the shuttle group).
94          - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
zeroing it on
95            the full variant rather than dropping a column.
96          - ``structural`` ? a guard whose value is in failure modes the golden corpus may
lack; NEVER
97            flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
``structural=True``).
98          """
99
100         name: str
101         group: str
102         columns: Tuple[str, ...] = ()
103         gate_knob: Optional[str] = None
104         structural: bool = False
105
106
107     @dataclass
108     class AblationConfig:
109         """How to run an ablation over a corpus.
110
111         - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
112           the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
carries
113           ``feature_names`` = all columns and the gates ON.
114         - ``signals`` ? the inventory to ablate.
115         - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
```

```
(LOGO).
        - ``iou`` ? temporal-IoU match threshold.
        - ``honest`` ? LOVO for learned variants (the number to trust).
        - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
        - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
        """

        full: Variant
        signals: List[Signal]
        granularity: str = "group"
        iou: float = 0.5
        honest: bool = True
        seed: int = 0
        p_threshold: float = 0.05


    @dataclass
    class AblationRow:
        """One signal's marginal-contribution row (?(s) + CI + sign test + C4 + a
verdict)."""

        signal: str
        group: str
        structural: bool
        dropped_columns: List[str]
        gate_knob: Optional[str]
        n: int
        delta_f1: float
        ci_low: float
        ci_high: float
        ci_excludes_zero: bool
        sign_wins: int
        sign_losses: int
        sign_p_value: float
        f1_full: float
        f1_ablated: float
        seg_ratio_full: float
        seg_ratio_ablated: float
        verdict: str
        structural_protected: bool

        def as_dict(self) -> Dict[str, Any]:
            return {
                "signal": self.signal,
                "group": self.group,
                "structural": self.structural,
                "dropped_columns": list(self.dropped_columns),
                "gate_knob": self.gate_knob,
                "n": self.n,
                "delta_f1": self.delta_f1,
                "ci_low": self.ci_low,
                "ci_high": self.ci_high,
                "ci_excludes_zero": self.ci_excludes_zero,
                "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
                              "p_value": self.sign_p_value},
                "f1_full": self.f1_full,
                "f1_ablated": self.f1_ablated,
                "segment_count_ratio_full": self.seg_ratio_full,
                "segment_count_ratio_ablated": self.seg_ratio_ablated,
                "verdict": self.verdict,
                "structural_protected": self.structural_protected,
            }


    @dataclass
```

```
    class AblationReport:
        """All signals' rows + the run's provenance ? a structured, JSON-able ablation
result."""

        granularity: str
        iou: float
        num_videos: int
        honest_lovo: bool
        full_set_signals: List[str]
        full_spec_hash: str
        label_set_hash: str
        rows: List[AblationRow] = field(default_factory=list)

        def sorted_rows(self) -> List[AblationRow]:
            """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
order)."""
            return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)

        def as_dict(self) -> Dict[str, Any]:
            return {
                "granularity": self.granularity,
                "iou": self.iou,
                "num_videos": self.num_videos,
                "honest_lovo": self.honest_lovo,
                "full_set_signals": list(self.full_set_signals),
                "full_spec_hash": self.full_spec_hash,
                "label_set_hash": self.label_set_hash,
                "rows": [r.as_dict() for r in self.sorted_rows()],
            }


    # ---------------------------------------------------------------------------- the driver


    def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
        """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
its
        ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
        mutates ``full``."""
        cols = set(columns)
        fn = full.feature_names
        ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
        overrides: Dict[str, Any] = {"feature_names": ablated_fn}
        if gate_knob == "min_crossings":
            overrides["min_crossings"] = 0
        elif gate_knob == "min_players":
            overrides["min_players"] = 0
        elif gate_knob is not None:
            raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
        dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
        return experiment.Variant.from_dict({**full.to_dict(), **overrides,
                                             "name": f"?{dropped_label}"})


    def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
                  sign_p: float, p_threshold: float) -> Tuple[str, bool]:
        """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.

        A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
        signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
```

```
worse on
236         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
237         wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
238         if structural:
239             return STRUCTURAL_PROTECTED, True
240         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
241         if ci_excludes_zero and sign_agrees:
242             return EARNS_KEEP, False
243         if sign_wins > sign_losses:           # leans toward "removal hurts" but evidence is
thin
244             return SUGGESTIVE, False
245         return INERT, False
246
247
248     def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
249                    signal: Signal) -> AblationRow:
250         """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
251
252         Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
253         variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
?(s).
254         Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
255         come for free."""
256         ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257         res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                                  honest_stats=True)
259         # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
260         # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
261         from backend.eval.eval_stats import paired_delta_ci
262         by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
263         by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
264         vids = [p["video"] for p in res["variant_a"]["per_video"]]
265         pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
266         st = pd["sign_test"]
267         sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
268         sign_p = float(st["p_value"])
269         verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
270                                        sign_wins, sign_losses, sign_p, cfg.p_threshold)
271
272         seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
273         seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
274
275         return AblationRow(
276             signal=signal.name,
277             group=signal.group,
278             structural=signal.structural,
279             dropped_columns=list(signal.columns),
280             gate_knob=signal.gate_knob,
281             n=int(pd["n"]),
282             delta_f1=float(pd["mean_delta"]),
283             ci_low=float(pd["ci_low"]),
284             ci_high=float(pd["ci_high"]),
285             ci_excludes_zero=bool(pd["ci_excludes_zero"]),
286             sign_wins=sign_wins,
287             sign_losses=sign_losses,
288             sign_p_value=sign_p,
289             f1_full=float(res["variant_b"]["aggregate"]["f1"]),
290             f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
291             seg_ratio_full=float(seg_full),
292             seg_ratio_ablated=float(seg_abl),
```

```
            verdict=verdict,
            structural_protected=protected,
        )


    def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
        """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
columns,
        OR the gate knob, OR the structural flag. First-seen group order is preserved."""
        order: List[str] = []
        by_group: Dict[str, List[Signal]] = {}
        for s in signals:
            if s.group not in by_group:
                order.append(s.group)
                by_group[s.group] = []
            by_group[s.group].append(s)
        out: List[Signal] = []
        for g in order:
            members = by_group[g]
            cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
            knob = next((m.gate_knob for m in members if m.gate_knob), None)
            structural = any(m.structural for m in members)
            out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
        return out


    def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
        """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).

        A pure driver: each row is one ``experiment.compare(without_s, full)``.
``granularity="group"``
        (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
        time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
        already persisted)."""
        if len(videos) < 1:
            raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
        if cfg.granularity not in ("signal", "group"):
            raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")

        inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
        rows = [ablate_one(videos, cfg, s) for s in inventory]

        # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
        full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
        from backend.eval.regression import gt_hash
        all_gts = [iv for vc in videos for iv in (vc.gts or [])]
        return AblationReport(
            granularity=cfg.granularity,
            iou=cfg.iou,
            num_videos=len(videos),
            honest_lovo=bool(full_eval["honest_lovo"]),
            full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
            full_spec_hash=cfg.full.spec_hash(),
            label_set_hash=gt_hash(all_gts),
```

```
345             rows=rows,
346         )
347
348
349     # ------------------------------------------------------------------ default
inventories
350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359         return [
360             Signal("shuttle.duration", "shuttle", ("duration",)),
361             Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362             Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363             Signal("person", "person", PERSON_COLUMNS),
364             Signal("audio", "audio", AUDIO_COLUMNS),
365         ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ------------------------------------------------------------------------ corpus
I/O
384
385
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals +
shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs
predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
390         mirroring ``experiment._feature_column``'s tolerance."""
391         videos: List[VideoCandidates] = []
392         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
```

```
            with open(path, encoding="utf-8") as f:
                d = json.load(f)
            cands = d["candidates"]
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_COLUMNS:
                features[fn] = [float(c["shuttle"][fn]) for c in cands]
            for fn in PERSON_COLUMNS:
                features[fn] = [float(c["person"][fn]) for c in cands]
            for fn in AUDIO_COLUMNS:
                features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
            gts = [(float(a), float(b)) for a, b in d["gts"]]
            videos.append(VideoCandidates(name=d["name"], intervals=intervals,
                                          features=features, gts=gts))
        return videos


    def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
        """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
        by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
        (distinct from the static Gate-A litmus)."""
        cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
        return experiment.Variant(name="full_learned", feature_names=cols,
                                  threshold=threshold, min_crossings=min_crossings)


    # ------------------------------------------------------------------------- table
rendering


    def format_table(report: AblationReport) -> str:
        """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
        rows = report.sorted_rows()
        header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
                  f"  ·  n={report.num_videos}  ·  IoU {report.iou}"
                  f"  ·  {'LOVO-honest' if report.honest_lovo else 'static'}"
                  f"  ·  {report.granularity.upper()}")
        cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
        widths = [18, 17, 18, 14, 5, 22]

        def _fmt_row(cells: Sequence[str]) -> str:
            return "  ".join(c.ljust(w) for c, w in zip(cells, widths))

        lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
        for r in rows:
            ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
            sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
            verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
            lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
                                   "yes" if r.ci_excludes_zero else "no", verdict]))
        return "\n".join(lines)


    # ------------------------------------------------------------------------- PDF
export


    def export_pdf(report: AblationReport, path: str) -> str:
        """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
marginal
        contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
        structural-protected). Reuses the muted booktabs aesthetic of
```

```
``scratch/quality_report.py``.
452
453         Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
GPU)."""
454         from reportlab.lib import colors
455         from reportlab.lib.enums import TA_CENTER, TA_LEFT
456         from reportlab.lib.pagesizes import A4
457         from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
458         from reportlab.lib.units import cm
459         from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table,
TableStyle)
460
461         navy = colors.HexColor("#1F3A5F")
462         black = colors.HexColor("#000000")
463         darkgrey = colors.HexColor("#333333")
464         light = colors.HexColor("#F0F0F0")
465         hairline = colors.HexColor("#BFBFBF")
466         row_alt = colors.HexColor("#F7F7F7")
467
468         base = getSampleStyleSheet()["Normal"]
469         title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold",
fontSize=16,
470                                leading=20, alignment=TA_CENTER, textColor=navy,
spaceAfter=4)
471         subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique",
fontSize=9.5,
472                                   leading=13, alignment=TA_CENTER, textColor=darkgrey,
spaceAfter=10)
473         h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11,
leading=15,
474                             textColor=navy, spaceBefore=10, spaceAfter=4)
475         body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9,
leading=12.5,
476                               alignment=TA_LEFT, textColor=black, spaceAfter=5)
477         cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2,
leading=10.5,
478                               textColor=black)
479         cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold",
fontSize=8.2,
480                                 leading=10.5, textColor=black)
481         head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
482                               leading=10.5, textColor=black)
483
484         rows = report.sorted_rows()
485         story: List[Any] = [
486             Paragraph("Signal Ablation &amp;amp; Feature-Enablement Report", title),
487             Paragraph(
488                 f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out
marginal "
489                 f"contribution &mdash; n={report.num_videos} golden matches, IoU
{report.iou}, "
490                 f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
491                 "Honest measured results only.",
492                 subtitle),
493         ]
494
495         full_set = ", ".join(report.full_set_signals)
496         story.append(Paragraph("Run provenance", h2))
497         story.append(Paragraph(
498             f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
499             f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
500             f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out
"
501             "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with
a "
```

```
            "bootstrap 95% CI and an exact paired sign test &mdash; a signal earns its keep
only "
            "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test
agreeing.",
            body))

        story.append(Paragraph("Marginal contribution per signal", h2))
        table_data: List[List[Any]] = [[
            Paragraph("Signal", head), Paragraph("Group", head),
            Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
            Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
            Paragraph("Over-seg", head), Paragraph("Verdict", head),
        ]]
        for r in rows:
            ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
            sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
            overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
            verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
            df1_style = cell_b if r.verdict == EARNS_KEEP else cell
            table_data.append([
                Paragraph(r.signal, cell), Paragraph(r.group, cell),
                Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
                Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no",
cell),
                Paragraph(overseg, cell), Paragraph(verdict, cell),
            ])
        col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm,
2.6 * cm]
        tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
        last = len(table_data) - 1
        tbl.setStyle(TableStyle([
            ("BACKGROUND", (0, 0), (-1, 0), light),
            ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
            ("TOPPADDING", (0, 0), (-1, -1), 4),
            ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
            ("LEFTPADDING", (0, 0), (-1, -1), 4),
            ("RIGHTPADDING", (0, 0), (-1, -1), 4),
            ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),
            ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
            ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
            ("LINEBELOW", (0, last), (-1, last), 1.0, black),
            ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
        ]))
        story.append(tbl)
        story.append(Spacer(1, 0.3 * cm))

        story.append(Paragraph("Verdict key", h2))
        story.append(Paragraph(
            "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI
clearing 0 "
            "and the sign test agreeing (the signal earns its keep). "
            "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but
the CI "
            "spans 0 (keep, watch as the corpus grows). "
            "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement
only after a "
            "trend across multiple runs, never on one wash at small n. "
            "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the
golden "
            "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported
honestly but "
            "never auto-flagged dead weight on 'no measured lift'.",
            body))

        doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
```

```
                                topMargin=1.8 * cm, bottomMargin=1.8 * cm,
                                title="Signal Ablation & Feature-Enablement Report",
                                author="badminton-highlight-indexer")
        doc.build(story)
        logger.info("wrote ablation PDF -> %s", path)
        return path


    # ---------------------------------------------------------------------------
CLI


    def main() -> None:
        ap = argparse.ArgumentParser(
            description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
                        "over experiment.compare + eval_stats; no new scoring math).")
        ap.add_argument("--features-dir", default="output",
                        help="dir of *_golden_features.json (the persisted golden
substrate)")
        ap.add_argument("--granularity", choices=("signal", "group"), default="group",
                        help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
        ap.add_argument("--litmus", action="store_true",
                        help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
                             "instead of the learned full-set ablation")
        ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
        ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
        args = ap.parse_args()

        videos = load_videos(args.features_dir)
        if len(videos) < 2:
            raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
                             f"(found {len(videos)})")

        if args.litmus:
            cfg = litmus_config()
            cfg.iou, cfg.seed = args.iou, args.seed
        else:
            cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
                                 granularity=args.granularity, iou=args.iou, seed=args.seed)

        report = run_ablation(videos, cfg)
        print(format_table(report))

        if args.json:
            with open(args.json, "w", encoding="utf-8") as f:
                json.dump(report.as_dict(), f, indent=2)
            print(f"\nwrote report JSON -> {args.json}")
        if args.pdf:
            export_pdf(report, args.pdf)
            print(f"wrote PDF report -> {args.pdf}")


    if __name__ == "__main__":
        main()

```

13. user (2026-06-16T03:51:11.915Z)

```
"""Experiment harness ? A/B + shadow + side-by-side for rally detection.

This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
no-regression gate, the pinnable classifier artifact, the persisted-candidate
substrate) already exists ? this module composes it. **No new scoring math lives
here**; everything defers to:

  - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
  - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
  - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
  - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
  - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
  - `backend.eval.calibration`    ? `pct_improvement`

The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
(per-frame signals ? run once on the GPU/WSL box, persisted into
`candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
given persisted candidate features it produces `List[Interval]` deterministically,
with no GPU and zero production risk. So most experiments are pure offline
re-scoring of stored data and never touch the served pipeline.

Three modes (`EXPERIMENT_HARNESS.md` §5):
  - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
    aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
    held-out loop, but variant-parameterised.
  - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
    variants agree / diverge (A-only, B-only, agreed). The divergences are what a
    human spot-checks (and what two highlight reels would show side by side).
  - the promotion gate stays `run_regression.py` + `regression_baseline.json`
    (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

Pure + offline + unit-tested. The only DB-touching helper is
`load_video_candidates`, which reads persisted candidates via
`Database.query_candidate_segments`.
"""
from __future__ import annotations

import json
import logging
import os
from dataclasses import dataclass, field
from datetime import datetime, timezone
from hashlib import sha1
from typing import Any, Callable, Dict, List, Optional, Sequence

from backend.eval.calibration import pct_improvement
from backend.eval.classifier import LogisticRegression
from backend.eval.gemini_refine import merge_overlaps
from backend.eval.metrics import Interval, match_intervals, score
from backend.eval.regression import gt_hash
from backend.eval.training import label_candidates
from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

logger = logging.getLogger(__name__)

# Where a comparison run appends one reproducible record. Tracked location (NOT under
# the gitignored output/) so the leaderboard survives a clean checkout, mirroring
# regression.DEFAULT_BASELINE_PATH.
DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
```

```
    _METRICS = ("precision", "recall", "f1", "mean_iou")


    class ExperimentError(ValueError):
        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
        score an unlabeled video with no pinned model, or a gate referencing an absent
        feature)."""


    # ---------------------------------------------------------------------- Variant


    @dataclass
    class Variant:
        """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

        A variant turns one video's persisted candidate windows + features into rally
        intervals. Every knob defaults to the **control** behaviour (no gate, no learned
        filter), so a variant that merely *carries* a new, disabled cue is byte-identical
        to control ? the core no-regression guarantee.

        Fields:
          - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
            for the leaderboard and for selecting the served path (the existing
            `segmenter_registry` + `IndexingConfig` do the actual selection in production).
            New cues are recorded here default-OFF.
          - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
            ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
            `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
            windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
          - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
            whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
            cue persists that feature).
          - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
            `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
            required for `compare_unlabeled` on a learned variant.
          - ``feature_names`` ? the persisted features the learned filter consumes. When set
            WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
            (honest) ? the recipe, not a pinned artifact.
          - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
            overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
        """

        name: str
        segmenter: str = "wasb_hybrid"
        config_overrides: Dict[str, Any] = field(default_factory=dict)
        cue_flags: Dict[str, bool] = field(default_factory=dict)
        min_crossings: int = 0
        min_players: int = 0
        classifier_model: Optional[str] = None
        feature_names: Optional[List[str]] = None
        threshold: float = 0.5
        merge_gap: float = 1.0
        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
        # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
        # failure where a fixed 0.5 zeroed out a small-candidate video.
        tune_threshold: bool = False          # pick the F1-best threshold on the train fold
        calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first

        def is_learned(self) -> bool:
            """True if this variant applies a learned classifier (pinned model or
```

```
recipe)."""
            return self.classifier_model is not None or bool(self.feature_names)

        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
```

```
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
```

```
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                       # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
```

```
to
306             report "how the shipped artifact does here", not for the promotion decision.
307           - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
308             train on the OTHER videos, predict the held-out one. The number to trust.
309           - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:
331                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332                 model: Optional[LogisticRegression] = _train(variant, others, iou)
333                 if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
334                     assert model is not None                       # _train never returns
None
335                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo:                          # honest=False ? one model over all
videos
337                 model = all_model
338             else:                                      # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None                  # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                      for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,
353             "per_video": per_video,
354             "aggregate": aggregate,
355             "predictions": predictions,
356         }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
```

```
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out


    def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
        """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).

        ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
        so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
        the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
        """
        a_f1 = [p["f1"] for p in eval_a["per_video"]]
        b_f1 = [p["f1"] for p in eval_b["per_video"]]
        return {
            "variant_a_ci": _ci_block(eval_a),
            "variant_b_ci": _ci_block(eval_b),
            "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
            "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                             "variant_b": _segmentation_block(videos, eval_b)},
        }


    def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
        """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
        golden corpus and reports the **B ? A** deltas, per video and in aggregate.

        The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
        learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
        `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.

        ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
```

```
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436         result: Dict[str, Any] = {
437             "iou": iou,
438             "label_set_hash": gt_hash(all_gts),
439             "num_videos": len(videos),
440             "variant_a": eval_a,
441             "variant_b": eval_b,
442             "delta": delta,
443             "delta_pct": delta_pct,
444             "per_video_delta": per_video_delta,
445         }
446         if honest_stats:
447             result["honest"] = honest_summary(videos, eval_a, eval_b)
448         return result
449
450
451     # ------------------------------------------------------------------ SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                           agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462           - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
463           - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
464           - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
465             the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
a
468         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469         `metrics.match_intervals` ? no new matching logic.
```

```
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
    the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
```

```
534         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535         with open(path, "w", encoding="utf-8") as f:
536             json.dump(entries, f, indent=2)
537         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                     row["variant_a"]["name"], row["variant_b"]["name"],
539                     (result.get("delta") or {}).get("f1", 0.0), path)
540         return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                               gts: Optional[List[Interval]] = None,
545                               feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
547
548         ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

14. assistant (2026-06-16T03:51:16.296Z)

The codebase facts are verified. Let me confirm the remaining anchors: the `fusion_golden.py` schema producer (the `name` field and feature-name constants), the existing skipif test pattern, and the `GOLDEN_DATA_SHARING.md` doc.

15. user (2026-06-16T03:51:17.569Z)

```
1       """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3       Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4       candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5       EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6       writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7       CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8       measured by the shipped experiment harness; person features enter only as feature
columns the
9       learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12          python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
```

```
    Each output `output/<name>_golden_features.json` is the replayable feature substrate ? re-run
    `fusion_compare` with different variants/thresholds forever without re-detecting on the GPU.
    """
    from __future__ import annotations

    import argparse
    import json
    import os
    import time
    from typing import Any, Dict, List, Optional

    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.pipeline.detectors import rally_gate
    from backend.pipeline.detectors import fusion_features as ff
    from backend.eval import distill_local
    from backend.eval.calibrate_wasb import load_golden_gts
    from backend.eval.training import label_candidates
    from backend.utils.run_telemetry import RunTelemetry

    # The 5 fps/resolution-invariant shuttle features distill_local already computes.
    SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
    # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
    PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)


    def _derive_video_path(spec: Dict[str, Any]) -> str:
        """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
        golden labels path (``X.rallies.csv`` -> ``X.MP4``, the golden-set naming convention)."""
        if spec.get("video_path"):
            return str(spec["video_path"])
        labels = str(spec["labels"])
        if labels.endswith(".rallies.csv"):
            return labels[: -len(".rallies.csv")] + ".MP4"
        raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv")


    def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
                      frame_skip: int = 3, iou: float = 0.5,
                      log_every_sec: float = 15.0,
                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
        """One golden video -> a feature record (shuttle + person features per candidate window).

        Returns ``{name, fps, frame_width, candidates:[{start,end,shuttle{},person{},label}], gts}``.
        `detector` (anything with ``detections_over_windows``) is injectable for GPU-free tests;
        when None a real `PersonDetector` is built (loads torchvision on first use).
        `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health snapshots
        from the detection progress callback so a hard kill leaves a forensic runlog.
        """
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        traj = str(spec["trajectory"])
        points = TrackNetRunner.parse_trajectory_csv(traj)
        intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
        gts = load_golden_gts(str(spec["labels"]))
        labels = label_candidates(intervals, gts, iou)
        # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic estimate
        # (parity with the shuttle net-crossing gate) rather than hard-coding an
```

```
orientation.
70          axis, net_px, _span = rally_gate.estimate_play_axis(points)
71          video_path = _derive_video_path(spec)
72
73          if detector is None:
74              from backend.pipeline.detectors.person_detector import PersonDetector
75              detector = PersonDetector({"indexing": {"use_gpu": True}})
76
77          # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78          # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79          n = len(intervals)
80          win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
```

16. user (2026-06-16T03:51:19.098Z)

```
tests\test_ablation.py:366:def _golden_present() -> bool:
tests\test_ablation.py:370:@pytest.mark.skipif(not _golden_present(),
tests\test_ablation.py:372:def test_litmus_reproduces_gate_a():
tests\test_ablation.py:398:@pytest.mark.skipif(not _golden_present(),
tests\test_ablation.py:400:def test_litmus_report_is_well_formed():
tests\test_served_gate_a_regression.py:66:_skip = pytest.mark.skipif(
```

17. user (2026-06-16T03:51:20.202Z)

```
34:SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35:# The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36:PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
56:    Returns ``{name, fps, frame_width, candidates:[{start,end,shuttle{},person{},label}],
gts}``.
62:    fps, fw = float(spec["fps"]), float(spec["frame_width"])
87:            telemetry.snapshot(done, total, phase=spec["name"])
104:        det_fw = det.get("frame_width") or fw
118:            "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
123:        "name": spec["name"], "fps": fps, "frame_width": fw,
160:            telem.event("video_start", spec["name"])
165:                telem.event("error", spec["name"], err=str(e))
171:            telem.event("video_done", spec["name"],
```

18. assistant (2026-06-16T03:51:25.901Z)

All anchors verified. Let me read the existing skipif test pattern exactly and confirm the `GOLDEN_DATA_SHARING.md` doc plus whether `requirements` already excludes/includes protobuf.

19. user (2026-06-16T03:51:27.324Z)

```
360     # ------------------------------------------------------------------------------ the
golden litmus
361
362     GOLDEN_DIR = os.path.join(
363         os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output")
364
365
366     def _golden_present() -> bool:
367         return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
368
369
370     @pytest.mark.skipif(not _golden_present(),
371                         reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
372     def test_litmus_reproduces_gate_a():
373         """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
374         result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
```

```
375         This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
376         where the golden substrate is absent; runs on the owner box where it is present."""
377         videos = ablation.load_videos(GOLDEN_DIR)
378         assert len(videos) == 6
379         cfg = ablation.litmus_config()
380         report = run_ablation(videos, cfg)
381         assert len(report.rows) == 1
382         row = report.rows[0]
383         assert row.signal == "gate_a"
384         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
385         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
386         assert row.sign_wins == 6 and row.sign_losses == 0
387         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
388         assert row.ci_low == pytest.approx(0.042, abs=0.01)
389         assert row.ci_high == pytest.approx(0.104, abs=0.01)
390         assert row.ci_excludes_zero is True
391         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
392         assert row.verdict == ablation.STRUCTURAL_PROTECTED
393         # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
394         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
395         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
396
397
398     @pytest.mark.skipif(not _golden_present(),
399                         reason="golden output/*_golden_features.json absent")
400     def test_litmus_report_is_well_formed():
401         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
402         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
403         assert isinstance(report, AblationReport)
404         assert report.num_videos == 6
```

20. user (2026-06-16T03:51:27.899Z)

```
docs\GOLDEN_DATA_SHARING.md
```

21. user (2026-06-16T03:51:27.921Z)

```
requirements.txt
training\requirements-train.txt
.claude\worktrees\bhi-execpolicy\requirements.txt
.claude\worktrees\bhi-execpolicy\training\requirements-train.txt
.claude\worktrees\agent-a771ff1d85e18f2ed\requirements.txt
.claude\worktrees\agent-a771ff1d85e18f2ed\training\requirements-train.txt
.claude\worktrees\local-noai\requirements.txt
.claude\worktrees\local-noai\training\requirements-train.txt
.claude\worktrees\docs-rally-quality\requirements.txt
.claude\worktrees\docs-rally-quality\training\requirements-train.txt
.claude\worktrees\tier0-eval\requirements.txt
.claude\worktrees\tier0-eval\training\requirements-train.txt
```

22. assistant (2026-06-16T03:51:32.145Z)

Notable: there's a `tier0-eval` worktree already ? that's likely the concurrent workflow on fixture mechanics. Let me check the requirements for protobuf and skim the GOLDEN_DATA_SHARING doc for the RALLY_GOLDEN_DIR seam and OneDrive store.

23. user (2026-06-16T03:51:33.202Z)

```
No matches found
```

24. user (2026-06-16T03:51:33.513Z)

Golden / GPU-Extractor Data Sharing ? GPU box ? dev sessions

Status: convention + plan (2026-06-15). Tracked in **#175**.
Purpose: a single cloud-synced location for the GPU-extracted golden artifacts (trajectory CSVs, `*_golden_features.json`, manifests, baseline snapshots) so the **GPU box** and **CPU dev/agent sessions** share the same files ? without committing large data to git.

Why this exists

The rally-quality measurement is split across two machines by capability:
- **`backend/eval/fusion_golden.py`** (shuttle+person feature **extraction**) needs the **videos**, the **trajectory CSVs**, the **`.rallies.csv` golden labels**, and a **GPU/WSL** stack ? only the GPU box can run it.
- **`backend/eval/fusion_compare.py`** / **`backend/eval/ablation.py`** (the LOVO **litmus**: ?F1, bootstrap CI, paired sign test) are **pure-CPU re-scoring** that read only the small `*_golden_features.json` ? any session (incl. a CPU dev container) can run them.

Today those artifacts sit in gitignored `output/` (per-machine) and scattered local folders (`...\Annotation Setup\Collect\Trajectories\`, `...\Golden Labelled\`). Nothing syncs them between machines, so a CPU session cannot see the GPU box's extraction. This wires that gap with OneDrive.

Canonical shared folder

`%USERPROFILE%\OneDrive\rally-annotator\golden-data\` ? `C:\Users\avidu\OneDrive\rally-annotator\golden-data\`

OneDrive is present + signed-in on both the GPU box and the dev machine under the same account, so it auto-syncs. Layout (date/size-tagged so a re-extraction is a new folder, never an overwrite):

```
golden-data/
  README.md
  features/<corpus-tag>/       *_golden_features.json     (small ? the key shared artifact)
  trajectories/<corpus-tag>/   *_predict.csv / *_wasb.csv (medium ? reusable across runs)
  manifests/                   *_lovo_manifest.json
  baselines/                   leaderboard / regression snapshots (or keep tracked in eval_baselines/)
```

`<corpus-tag>` = e.g. `2026-06-15-n6` (date + corpus size). **Do NOT** put raw videos here (tens of GB) ? keep those local to the GPU box.

The GPU ? CPU workflow

1. **GPU box** (videos + CSVs + labels present):
   ```
   python -m backend.eval.fusion_golden --manifest <manifest> \
     --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-data\features\2026-06-15-n6"
   ```
   (or extract to local `output/` then copy the `*_golden_features.json` into the shared folder).
2. OneDrive syncs the JSONs to the dev machine automatically (small files, seconds).
3. **CPU dev/agent session** points the analysis at the shared folder:
   ```
   python -m backend.eval.fusion_compare --features-dir "...\golden-data\features\2026-06-15-n6" --record
   python -m backend.eval.ablation       --features-dir "...\golden-data\features\2026-06-15-n6" --granularity group
   ```
   ? real person-feature ?F1 / CI / sign test. Copy the JSONs into the repo's `output/` to also light up the `skipif` golden-litmus tests (`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`).

```
44      ## Optional code ergonomics (in #175, not required)
45      - A `RALLY_GOLDEN_DIR` env var (default `output/`) honored by `fusion_golden` /
`fusion_compare` / `ablation` so neither machine hardcodes the path. (The CLI `--out-dir` /
`--features-dir` flags already make this a convention; the env var is just ergonomics.)
46      - A tiny `scripts/sync_golden.py` to copy `output/*_golden_features.json` ? the shared
folder.
47
48      ## Guardrails
49      - The shared folder lives **outside the repo** ? never committed. Keep `output/`
gitignored and do **not** symlink the shared folder into a tracked path.
50      - The features derive from the **Proprietary** corpus ? keep the OneDrive folder
**private** (no public share links).
51
52      See also: `docs/NEXT_STEPS.md` (the "Next-session pickup" anchor),
`docs/MULTI_SIGNAL_FUSION_PLAN.md` (P3), `docs/TRACKNET_WSL_SETUP.md` (the GPU/WSL detector
setup).
53
```

25. assistant (2026-06-16T03:51:38.102Z)

All facts verified. Let me quickly check the `tier0-eval` worktree to understand what the concurrent fixture-mechanics workflow has produced, so I build on it rather than collide.

26. user (2026-06-16T03:51:39.352Z)

```
+ bhi-execpolicy
+ docs/rally-quality-research
  feat/highlights-runner
+ feat/local-no-ai-segmenter
+ feat/tier0-rally-seg-eval
  fix/compiler-dedup-overlapping-segments
  fix/gemini-per-video-chunk-dir
  fix/wasb-test-mocks-readability-preflight
  fix/wsl-unreadable-source-preflight
* master
+ perf/wasb-fast-frames
  worktree-agent-a771ff1d85e18f2ed
  remotes/origin/HEAD -> origin/master
  remotes/origin/docs/pmf-field-signals
  remotes/origin/docs/rally-quality-research
  remotes/origin/feat/highlights-runner
  remotes/origin/feat/local-no-ai-segmenter
  remotes/origin/fix/compiler-dedup-overlapping-segments
  remotes/origin/fix/gemini-per-video-chunk-dir
  remotes/origin/fix/wasb-test-mocks-readability-preflight
  remotes/origin/fix/wsl-unreadable-source-preflight
  remotes/origin/master
  remotes/origin/perf/wasb-fast-frames
---WORKTREES---
C:/Users/avidu/Projects/badminton-highlight-indexer
960c00d [master]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed
7e0881a [perf/wasb-fast-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy
5cb7bbf [bhi-execpolicy]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/docs-rally-quality
877ab1f [docs/rally-quality-research]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/local-noai
3ea7ad4 [feat/local-no-ai-segmenter]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier0-eval
960c00d [feat/tier0-rally-seg-eval]
```

27. user (2026-06-16T03:51:39.637Z)

No files found

28. assistant (2026-06-16T03:51:50.661Z)

The `feat/tier0-rally-seg-eval` worktree is at the same commit as master (960c00d) with no fixtures built yet ? the concurrent mechanics workflow hasn't landed code. My design composes cleanly with it as a sibling. I have everything I need. Let me synthesize the unified architecture.

The verified facts confirm every load-bearing claim in the design components:
- `metrics.py:25,124-125` ? IoU and start/end errors are pure differences (translation-invariant) ?
- `fusion_compare.py:50` ? `d["name"]` is used only as an opaque `VideoCandidates` label ?
- `ablation.py:403` ? audio columns already default to 0.0 (additive-column tolerance) ?
- `eval_baselines/heuristic_lovo_n6.json:8-13` ? the live de-attribution leak (real player/source names) ?
- `regression.py:33,37-40` ? committed-baseline precedent + `gt_hash` ?
- `experiment.py:148,180-192` ? `spec_hash()` + `_feature_column` 0.0 tolerance ?
- `tests/test_ablation.py:366-371` ? the `_golden_present()` skipif pattern ?
- No protobuf/cryptography/sops in requirements yet ?

I'm now ready to deliver the unified architecture via StructuredOutput.

29. user (2026-06-16T03:54:25.645Z)

Structured output provided successfully

30. assistant (2026-06-16T03:54:42.562Z)

I've delivered the unified SEALED-FIXTURES architecture via StructuredOutput. Summary of what was verified and synthesized:

Verified against the live codebase (every load-bearing claim):
- `backend/eval/metrics.py:25,124-125` ? IoU and start/end errors are pure differences, so the time-origin reset is provably zero-cost.
- `backend/eval/fusion_compare.py:50` ? `d["name"]` is used only as an opaque `VideoCandidates` label, so aliasing it is harness-transparent.
- `backend/eval/ablation.py:403` ? audio columns already default to 0.0, confirming the additive-column tolerance.
- `backend/eval/experiment.py:148,180-192` ? `Variant.spec_hash()` and `_feature_column`'s 0.0 default exist as claimed.
- `backend/eval/regression.py:33,37-40` ? `eval_baselines/` committed precedent + `gt_hash`.
- `tests/test_ablation.py:366-371` ? the `_golden_present()` skipif pattern to generalize.
- `eval_baselines/heuristic_lovo_n6.json:8-13` ? **confirmed live de-attribution leak** (real names: adarsh_avi_singles, kushagra_singles, mahadevpura_singles, gbaaddy). This is Phase-1 remediation, not hypothetical.
- No `protobuf`/`cryptography`/`sops` in `requirements.txt` yet ? protobuf (Apache-2.0) clears the licensing guardrail when added.
- `feat/tier0-rally-seg-eval` worktree sits at master HEAD with no fixtures landed ? the concurrent mechanics workflow composes cleanly as a sibling; no collision.

The merge in one breath: non-attributability comes from deletion+relativization at the source (survives the analog hole, protects even keyed contributors), while uselessness-to-the-no-key-public comes from minimization plus optional encryption ? two different layers, never conflated. Tier-0 public/unencrypted/no-key runs the shuttle-only path unconditionally; Tier-1 key-gated (out-of-repo+token preferred, SOPS+age fallback) runs the full corpus and skips cleanly when absent. Six small-PR phases, with counsel sign-off as an explicit gating precondition the architecture assumes rather than satisfies.

One flag for the owner that surfaced during verification: the `heuristic_lovo_n6.json` name leak is already committed to `master` ? it should be the first PR regardless of whether the rest of the rollout proceeds.